University of Padova

Department of Mathematics

Master's Degree in Data Science

Super Resolution and Deblurring of Hyperpanoramic Images Using Neural Networks

Supervisor

Wolfgang Erb

University of Padova

Master Candidate

Marco Barbierato

2097844

Academic Year

2023-2024

Abstract

In this thesis we studied the application of Deep Neural Networks to the problem of Super Resolution of images, which consists in enhancing their spatial resolution and perceptual quality; we focused on images captured by a hyperpanoramic lens which presents spatially variant blur caused by its point spread function. We investigated two different architectures, CNNs and Transformers, together with specific network designs particular for the task of Super Resolution such as adversarial networks, and variants of the Adam optimization algorithm for training of deep networks.

We modified an existent pipeline for generating supervised training data to let our networks learn the spatially variant degradations of the panoramic images, and evaluated their performance with a no-reference image quality metric. Both architectures perform well in Super Resolution, with Transformers generally outperforming CNNs; by attribution analysis, we understood that Transformers have in fact a larger receptive field. We also found that adversarial training, while improving image quality, is more susceptible to the generation of artifacts.

Contents

Listing of figures

Listing of tables

List of Algorithms

Listing of acronyms

CDF . . . . . Cumulative Distribution Function

(FF)NN . . . . . (Feed Forward) Neural Network

GAN . . . . . Generative Adversarial Network

LAM . . . . . Local Attribution Map [1]

LR, HR . . . . . Low/High Resolution

PDF . . . . . Probability Distribution Function

PSNR . . . . . Peak Signal to Noise Ration

RDB . . . . . Residual Dense Block

RRDB . . . . . Residual in Residual Dense Block

RSTG . . . . . Residual Swin Transformer Block

SGD . . . . . Stochastic Gradient Descent

SSIM . . . . . Structural Similarity Index Measure

Swin . . . . . Shifted-Windows Transformer

SR . . . . . Super Resolution

‘

1 Introduction

The problem of *Super Resolution* (SR) consists of enhancing the spatial resolution of a given low-resolution image. Although it is possible to do so by naively interpolating it, this essentially does not add any meaningful information to the image, and neither does it improve its perceptual quality substantially. Correlated problems to the one of SR are the ones of *deblurring* and *denoising*, where it the recovery of a high-quality image is attempted starting from a degraded version, respectively, with blur or noise applied. These three problems can be considered difficult because they are ill-defined in nature, i.e. there is no single one-to-one correspondence between any image and its super-resolved, deblurred, or denoised version.

Classical quantitative methods for solving the SR problem usually assume that multiple instances of a low-resolution picture are available [2] and that the content of the high-resolution image can be recovered by combining the information contained in these samples; for the recovery of images degraded with spatially invariant blur and independent Gaussian noise, iterative methods have also been extensively studied.

Data-driven methods have similarly achieved great success in all of the above problems, also thanks to recent advances on the computational side that eased their training procedure. Specifically for the task of SR, Neural Networks (NN) trained in a supervised manner have good performance in *single-image* SR (i.e. the network only requires one image input), but also in the tasks of single-image deblurring and denoising. These networks are trained in a supervised manner starting with a dataset of high-resolution images which get downsampled (or blurred, or noise is added) to generate their low-resolution counterpart which together constitute the input-output pairs used for supervised learning.

In this work, we attempted the use of NNs for the SR of images captured by a particular hyperpanoramic lens called PANCAM[3] (PANoramic CAMera), which captures images that present fairly complicated and spatially variant degradations; these obviously render the problem of SR even more difficult, as one also needs to treat the blur caused by the optical system's point spread function. To let our NN learn to invert these degradations and also super-resolve

the images, we modified an existing degradation pipeline [4] used for the SR of real-world images to be able to also represent PANCAM degradations. However, as the SR of these images is fairly complicated, we decided to start by considering first the problem of simple SR (with no other degradations), then the problem of SR with spatially invariant blur, and finally attacking the spatially variant nature of the degradations in PANCAM images.

The thesis is organized as follows.

In Chapter 2 we start by recalling the basic setting of the supervised machine learning problem and its solution using feed-forward linear neural networks, and the process of training them via gradient optimization that leads to the rules of backpropagation; we end by explaining how this is generalized to networks composed of more complex operations with backward mode automatic differentiation, and with some comments about training very deep neural networks.

In Chapter 3 we introduce the main two architectural backbones that are used for neural networks with image inputs: convolutional layers and the Swin Transformer [5] (an improvement of the Vision Transformer [6]; then we explain the loss functions and validation metrics that are implemented for training and evaluation networks with image output. We also explain the concept of adversarial training [7] and its usefulness for the task of SR; we conclude the chapter by explaining the Adam [8] optimization algorithm with the addition of Nesterov Momentum and decoupled weight decay and its use for training very deep networks.

In chapter 4 we formally define the problem of SR and describe the challenges in super-resolving the particular images taken by PANCAM, explaining the approach we followed by organizing our work in tasks of increasing complexity; we introduce the pipeline that is used for generating supervised training pairs, including our modification to include spatially variant blur for PANCAM images; we then describe the neural network architectures we chose for these tasks (one convolutional, one transformer-based); finally, we present our results and compare the performance of the two architectures, highlighting the respective advantages and disadvantages. We finish with some final remarks and suggestions about possible improvements to our work.

2

Machine Learning and Neural Networks

In this chapter we start by defining the problem of supervised machine learning and explain its solution in terms of feed-forward neural networks, which are the starting point for deeper and complicated neural networks. We then briefly recall how the problem is approached and solved with gradient based iterative methods, and then explain the rules of back-propagation for linear networks and its generalization of backward mode automatic differentiation. When not indicated otherwise, the main text we used as reference for this chapter is [9].

2.1 Supervised machine learning

Supervised Machine Learning, in its most general formulation, deals with the problem of learning a unknown function $F : X \to Y$, where X and Y are respectively the sets of input and output of our problem. This function can be interpreted as a joint probability distribution p_{xy} on these two sets, such that we are not restricted to solely deterministic functions.

We then define a function $f : X \to Y$, taken from a chosen subset $\mathcal{F}$ of the functions from X to Y. We call this function the *neural network*, and the task of this function is that of approximating our initial target function F, or equivalently, the associated probability distribution. The subset $\mathcal{F}$ from which we choose our function is called the *hypothesis space*.

We moreover assume we have defined a function $L : Y \times Y \to \mathbb{R}$, called the *loss function*, which we take to be a measure of similarity on the set Y (how this is chosen depends on the specific problem), such that given two elements $y_1, y_2 \in Y$, the lower $L(y_1, y_2)$ is, the more similar the two elements are.

Our task then can be formulated like so: we want to find the function $f^* \in \mathcal{F}$ that minimizes the expected value of the loss function with respect to the probability distribution given by F.

$$f^* = \arg\min_{f \in \mathcal{F}} \mathbb{E}_{p_{xy}}[L(f(x), y)] \tag{2.1}$$

In this manner the function f^* can be taken as an approximation of our target function F; how good the above approximation is depends on the hypothesis space $\mathcal{F}$.

When dealing with practical problems the function F is unknown, and so is the probability distribution p_{xy}. As a result the above expected value can not be computed analytically. However, we assume we are given the possibility of sampling repeatedly and independently from this distribution such that we can build a set of input-output tuples

$$S = \{(x_i, y_i), \mid x_i \in X,\ y_i \in Y\ i = 1, ..., N\} \tag{2.2}$$

which we call the *training set*. Then, the above expected value can be estimated empirically as an average over the sampled training data, so the minimization problem becomes

$$\arg\min_{f \in \mathcal{F}} \frac{1}{|S|} \sum_{(x,y) \in S} L(f(x), y) \tag{2.3}$$

This approach is called *empirical risk minimization*, as above we are estimating the expected value with respect to the probability p_{xy} with the empirical measure given by a realization of sampling from said distribution.

The successful completion of a practical problem of this type, given a training set S, depends on the class $\mathcal{F}$ of functions that we consider for our approximation, the loss function, and the method used for solving the minimization task, which need to be carefully selected considering the problem at hand. For example, if we want to approximate a function $F : \mathbb{R} \to \mathbb{R}$ that we know to be linear or almost, an appropriate class of functions would be the real linear functions $\mathcal{F}_{lin}$; an appropriate loss function would be the square error $L(y_1, y_2) = (y_1 - y_2)^2/2$, so that our problem becomes

$$\min_{f \in \mathcal{F}_{lin}} \frac{1}{|S|} \sum_{(x,y) \in S} L(f(x), y) = \min_{\theta_1, \theta_2 \in \mathbb{R}} \frac{1}{2|S|} \sum_{(x,y) \in S} (\theta_1 x + \theta_2 - y)^2 \tag{2.4}$$

This is the well known ordinary least squares problem, and its solution parameters θ_1, θ_2 can be found analytically in terms of the input-output tuples of S.

In more complex problems however, it is very unlikely that the function F we want to approximate is simply linear.

2.1.1 Feed Forward Neural Networks

Feed Forward Neural Networks (FFNN, or NN) are a specific class of neural networks that can be used when the unknown function F we want to approximate might be extremely complex. We describe here their structure. We suppose from now on that the set X, Y are real vector spaces of certain dimensions n and m.

A FFNN is composed of a series of *layers* stacked one after another. A single layer, which

we call g_i, takes as input a vector $x \in \mathbb{R}^{n_i}$, and is defined by a matrix of parameters W_i of dimensions $m_i \times n_i$, plus usually a parameter $b_i \in \mathbb{R}^{m_i}$ called the bias; these construct an affine transformation of the input vector by $W_i \cdot x + b_i$, which is composed with a function h_i called the *activation function* to give the action of the layer g_i:

$$g_i(x) = h_i(W_i \cdot x + b_i) \tag{2.5}$$

The activation h is taken to be a simple but non-linear function that is used to let the NN inherit its non-linear nature, and is applied component wise to the vector it takes as argument. An example is RELU, defined to be

$$\mathrm{RELU}(x) = \max(x, 0) \tag{2.6}$$

A FFNN is then made as a composition of a certain number d of layers, called the depth of the network, as above:

$$\mathrm{NN(x)} = g_d \circ \cdots \circ g_1(x) \tag{2.7}$$

where the dimensions of the matrices of each layer i match so that their composition is properly defined. Given z_l to be the output of layer g_l, the dimension of z_l defines the *width* of the layer.

FFNN have been proved in particular cases to be universal function approximators, such as in the case of arbitrary depth [10] or arbitrary width [11], and with particular choices of activation functions. Informally this means that, given a function F between real vector spaces satisfying certain regularity assumptions, in the simplest case continuity, there exist a FFNN that can approximate it to any level $\varepsilon > 0$, with respect to some functional norm. The family of these networks can then be an attractive choice for the hypothesis space $\mathcal{F}$ defined above, particularly when we are unable to assume any particular structure for the unknown function F. The empirical minimization problem 2.3 in this case becomes

$$\underset{\{W_i, b_i\}}{\arg\min} \frac{1}{|S|} \sum_{(x,y)\in S} L(\mathrm{NN}(x), y) \tag{2.8}$$

since the parameters of our network are given by the matrices W_i and biases b_i. Given the complicated nature of a general neural network, even with a simple loss function L an analytical solution for this problem as in the case of ordinary least squares cannot be found. Thus, the problem is usually solved with iterative methods, most commonly gradient based methods.

2.2 Optimization

We are then interested in solving the minimization problem in equation (2.8). We always assume that the loss function L we are dealing with is at least differentiable.

We make explicit the dependence of the function that we want to minimize by the relevant

parameters, in the case of a FFNN the parameter matrices W_i and biases b_i. We collectively refer to the set of parameters that compose our NN by θ, so we can write our problem as

$$\min_{\theta} \frac{1}{|S|} \sum_{(x,y)\in S} L(\mathrm{NN}_\theta(x), y) := \min_{\theta} L(\theta) \tag{2.9}$$

With a differentiable function to minimize it's possible to solve this problem with a gradient based method. These start with an initial parameter θ_0 and iteratively update it by approximating the target function $L(\theta)$ with its Taylor expansion truncated at the first linear term, minimizing this approximation instead of the original function; the direction of greatest descent from θ_0 turns out to be minus the gradient of the function at this point, $-\nabla_\theta L(\theta_0)$, such that one can update the parameters with the rule

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla_\theta L(\theta_t) \tag{2.10}$$

where α_t is called the *stepsize* and is a chosen parameter that controls how far to descend at each step. The parameters are updated until they meet some chosen criteria that assures their convergence.

Under certain conditions on the function to be minimized $L(\theta)$, particularly convexity of the function and Lipschitz continuity of its gradient, one can prove that a gradient method converges to a local minimum of L [12], which in the case of convexity is also a global minimum. However, when one works with even simple neural networks, the function to be minimized is not convex in the parameters, and might in fact have many local minima: in this case one hopes that the problem at hand is well posed enough that even if there are many minima they minimize similarly the objective function; empirically, for data science problems, it is usually the case. Moreover, one usually modifies simple gradient descent such that it is able to avoid non-desirable minima. While gradient methods are born out of optimizing a linear approximation of the objective function, second order methods update similarly the parameters by minimizing instead a quadratic approximation; however, these methods requires a computation of the Hessian matrix of the objective function, which for problems with a many parameters becomes too expensive.

In our particular case, the gradient of the loss function turns out to be a sum over each tuple of the training set S; in data science problem this set might be very big, so that each update iteration becomes computationally expensive; *stochastic gradient*, in its usual formulation, updates the parameters by computing the gradient with respect to only one sample from S: this is comparatively very fast, and by virtue of having higher variance it manages to avoid stopping at non-optimal local minima during iteration. *Mini-batch gradient descent* is a compromise between these two approaches where at each iteration a subset of a certain *batch size* b of S is chosen and the gradients are computed and averaged only over this set. We report the algorithm in 2.1.

Algorithm 2.1 Mini-Batch Stochastic Gradient.

Require: α_t: stepsizes; Initialize θ_1 to a chosen value.
  while θ_t has not converged:
    $t \leftarrow t + 1$
    Choose a mini-batch of samples $\{(x_i, y_i) \in S \mid i = 1, ..., b\}$
    $\theta_{t+1} \leftarrow \theta_t - \alpha_t \frac{1}{b} \sum_{i=1}^{b} \nabla L_i(\theta)$, where $L_i(\theta) := L(NN_\theta(x_i), y_i)$
  end while

2.3 Backpropagation

To make an update of a mini-batch gradient method, one needs to compute the partial derivatives of the loss function given a certain training sample with respect to each parameter of the network:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \frac{1}{b} \sum_{i=1}^{b} \nabla L(NN_\theta(x_i), y_i) \tag{2.11}$$

For FFNN, this is done by repetitively exploiting the chain rule. Using the notation of (2.5) and (2.7), we want to compute the gradients with respect to each parameter matrix and bias (for simplicity, we suppress the arguments of the partial derivatives),

$$\frac{\partial L_i}{\partial W_j}, \quad \frac{\partial L_i}{\partial b_j} \tag{2.12}$$

For each layer l, we call z_l the output of the linear operation and a_l the resulting composition with the layer's activation function, such that a layer is given by

$$a_l = h_l(z_l) = h_l(W_l \cdot a_{l-1} + b_l) \tag{2.13}$$

By using the chain rule, it's straightforward to compute the partial derivatives of the parameters of the last layer:

$$\frac{\partial L_i}{\partial W_d} = \frac{\partial L_i}{\partial a_d} \frac{\partial a_d}{\partial W_d} = \frac{\partial L_i}{\partial a_d} \frac{\partial a_d}{\partial z_d} \frac{\partial z_d}{\partial W_d} \tag{2.14}$$

Where each term is then

$$\frac{\partial L_i}{\partial a_d} = L'(a_d, y_i), \quad \frac{\partial a_d}{\partial z_d} = h'_d(z_d), \quad \frac{\partial z_d}{\partial W_d} = a_{i-1}^T \tag{2.15}$$

For the bias term, the only difference becomes the last derivative, which is $\frac{\partial z_d}{\partial b_d} = 1$.

One can use the chain rule multiple times to get the derivatives of the weights of any layer l:

$$\frac{\partial L_i}{\partial W_l} = \frac{\partial L_i}{\partial a_d}\frac{\partial a_d}{\partial z_d}\frac{\partial z_d}{\partial a_{d-1}} \cdots \frac{\partial z_{l+1}}{\partial a_l}\frac{\partial a_l}{\partial z_l}\frac{\partial z_l}{\partial W_l} \tag{2.16}$$

where, in addition to the previous terms, we have

$$\frac{\partial z_l}{\partial a_{l-1}} = W_l^T \tag{2.17}$$

One can define the auxiliary vector δ_l

$$\delta_l = \frac{\partial L_i}{\partial a_d}\frac{\partial a_d}{\partial z_d}\frac{\partial z_d}{\partial a_{d-1}} \cdots \frac{\partial z_{l+1}}{\partial a_l}\frac{\partial a_l}{\partial z_l} \tag{2.18}$$

so that

$$\frac{\partial L_i}{\partial W_l} = \delta_l a_{l-1}^T \qquad \frac{\partial L_i}{\partial b_l} = \delta_l \tag{2.19}$$

and δ_l at each layer can be computed recursively starting from the last:

$$\delta_{l-1} = \frac{\partial z_l}{\partial a_{l-1}}\frac{\partial a_{l-1}}{\partial z_{l-1}}\delta_l = W_l^T h'_{l-1}(z_l)\delta_j \tag{2.20}$$

Back-propagation makes use of this recurrence relationship to compute the derivatives of the network parameters by starting from the last layer, and sequentially looping through them until the first one. See algorithm 2.2.

Algorithm 2.2 Algorithmic Backpropagation for a FFNN.

for each input-ouput (x_i, y_i) in a mini-batch:
  Compute the output of the network, $a_d = NN(x_i)$, tracking the variables $a_l,\ z_l$.
  $\delta_d = L'(a_d, y_i)h'_d(z_d)$
  for each layer $l = d, d-1, ..., 1$:
    $\frac{\partial L_i}{\partial W_l} = \delta_l a_{l-1}^T, \quad \frac{\partial L_i}{\partial b_l} = \delta_l$
    $\delta_{l-1} = W_{l-1}^T h'_{l-1}(z_{l-1})\delta_l$
  end for
end for
Update each network parameter via gradient descent.

2.3.1 Automatic Differentiation

The above back-propagation algorithm for the computations of parameter gradients works in the case of a neural network solely structured as the composition of a certain number of linear layers; however, one might want to consider an architecture with more elaborate operations or

more complicated control flow, such as if-else statements and loops. Automatic differentiation is a procedure that permits the algorithmic computation of partial derivatives of a function that is composed of elementary operations, i.e. a set of operations whose partial derivatives with respect to each input are assumed to be known and algorithmically computable. Here we report the exposure of automatic differentiation given by [13] and [14].

Given a function $f : \mathbb{R}^n \to \mathbb{R}^m$, the process of automatic differentiation starts with defining the required number of variables to keep track of: input variables $v_0, ..., v_{-(n-1)} = x_1, ..., x_n$, a certain number u of intermediate variables $v_1, ..., v_u$ which are built starting from the inputs (and other intermediate variables) with elementary operations, and the final output variables $v_{u+1}, ..., v_{u+m} = y_1, ..., y_m$. The relationships between inputs and outputs is then represented by a directed acyclic computational graph dependent on the computation rules of f, where the leaves are the inputs, the roots are the outputs, and an edge directed from v_i to v_j indicates that the variable v_i is an input to the elementary operation that computes v_j.

There are then two modes for computing the derivatives of f with respect to each variable v_i: forward mode and reverse mode, the latter of which is a generalization of back-propagation and which we describe now. It is composed of two phases:

1. A forward phase, where starting from the input we build the computational graph of f by computing all the intermediate variables following the logic and elementary operations that compose our function.

2. A backwards phase, where the derivatives with respect to each variable are computed, starting from the output variables.

For the backwards phase, each variable v_i is accompanied by an adjoint variable $\bar{v}_i^k$ which is defined to be the partial derivative of an output with respect to the variable itself:

$$\bar{v}_i^k = \frac{\partial y_k}{\partial v_i} \tag{2.21}$$

and starting the outputs y_k, which have trivial adjoints, one computes the predecessors' adjoints in the computational graph using the chain rule, in a backwards fashion:

$$\bar{v}_i^k = \sum_{\substack{j \text{ is a} \\ \text{successor} \\ \text{of } i}} \frac{\partial v_j}{\partial v_i} \bar{v}_j^k \tag{2.22}$$

where the term $\frac{\partial v_j}{\partial v_i}$ is then the partial derivative of one of the elementary operations.

For a FFNN the elementary operations are the application of an affine linear transformation $Wx + b$ and the application of an activation function $h(z)$. The computational graph is the same at each forward phase, and for a single layer can be represented as in figure (2.1). To

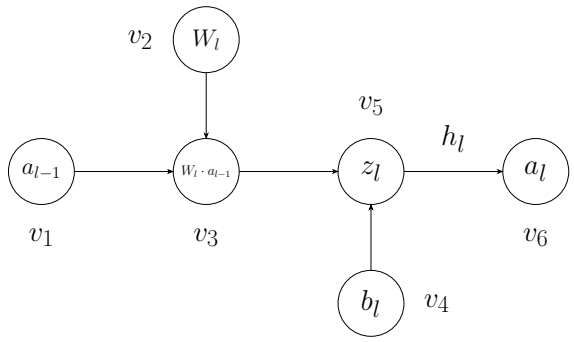


Figure 2.1: Computational graph for a linear layer in a FFNN.

compute for example the derivative $\frac{\partial a_l}{\partial a_{l-1}}$, one has

$$\frac{\partial a_l}{\partial a_{l-1}} = \bar{v}_1 = \frac{\partial v_3}{\partial v_1}\bar{v}_3 = \frac{\partial v_3}{\partial v_1}\frac{\partial v_5}{\partial v_3}\bar{v}_5 = \frac{\partial v_3}{\partial v_1}\frac{\partial v_5}{\partial v_3}\frac{\partial v_6}{\partial v_5}\bar{v}_6 = W_l^T \cdot 1 \cdot h_l'(z_l) \cdot 1 \tag{2.23}$$

so that one essentially recovers the back-propagation algorithm.

The python package Pytorch [15] that we use for our implementations offers an automatic differentiation backbone with dynamic computational graphs, such that one can even change the control flow of the network at each iteration and a different graph is generated at each forward phase.

2.4 Deep Neural Networks

We informally call a neural network *deep* if its depth is much larger than its width.

Deep neural networks have in particular a very large number of learnable parameters, and possibly layers more complicated than a linear transformation as in 2.5 as we will see the the next chapter. This produces a number of problems:

1. The loss landscape defined by the minimization problem 2.9 is extremely complicated, with many non-optimal local minima and possibly regions of very small gradient $\nabla L(\theta)$.

2. As the input of the network passes through a high number of layers, if the respective weights are not carefully tuned there is the possibility of its values to explode towards infinity or vanish towards 0. Similarly, in the backwards pass of back-propagation/automatic differentiation the same problem holds for the gradients.

3. A bigger network is also able to represent a more wide set of functions, and thus leads to the problem of *overfitting* on the training data, i.e. the network learns very well the empirical distribution of the training dataset S but does not perform as well on samples external to the same distribution.

The latter problem is solved by keeping a small subset of S (or another different set from the same distribution) and using it not for training but for *validating* the performance of the NN to avoid over-fitting.

The first two problems and in general the training instability that comes with training deep networks can be mitigated by using more sophisticated optimization methods, the normalization of layers through the network, particular architectural choices, and reasonable initialization for the network weights. We will explore these devices in the following chapters.

3

Neural Networks for Computer Vision

In this chapter we introduce the two main neural network architectural backbones used for working with images: convolutional networks and (a variation of) vision transformers; we also explain the concept of adversarial training and how it is used in the context of supervised learning. We then explain the main loss functions and validation metrics used for image outputs, and finish with a description of the NAdam [16] optimization algorithm, used to ease the training of very deep neural networks.

3.1 Convolutional Neural Networks

We have introduced FFNN in the previous section, but these have the problem that since every neuron is connected to each other, the number of parameters increases drastically as both the input size and depth of the network get bigger. Moreover, images have features that are particularly spatially invariant, and FFNN do not take advantage of these.

Convolutional neural networks can be used as a solution to these problems when dealing with image tasks. These make use of the (discrete) convolution operation between an image and a *kernel*: let $I(i,j) \in \mathbb{R}^{H\times W}$ be an one channel image and $K \in \mathbb{R}^{k\times k}$ be a kernel (for simplicity we assume it's a square matrix); the output of the convolution between I and K is indicated by $I * K$ and is given by

$$(I * K)(i,j) = \sum_{1\leq a,b\leq k} I(i-a, j-b)K(a,b) \tag{3.1}$$

The output of a convolution in when in the context of a neural network is sometimes referred to as a *feature map*. An equivalent operation to the one of convolution, which is the one usually

implemented in machine-learning libraries, is the *cross-correlation*, given by

$$(I * K)(i, j) = \sum_{1 \leq a,b \leq k} I(i + a, j + b)K(a, b) \tag{3.2}$$

Cross correlation is equal to convolution where the values of the kernel K are flipped symmetrically around its axes.

A *convolutional layer* is defined in terms of cross-correlation, and this can be used as a substitute for a linear layer in a feed forward neural network, where the parameters are the elements of the kernel. We suppose now that we are working with an input image I with C_{input} channels, and we want an output feature map O with C_{output} channels. Then, the j-th channel of the output O of the convolutional layer is given by

$$O(C_j) = b_j + \sum_{c=1}^{C_{input}} K_j(c) * I(c) \tag{3.3}$$

where $I(c)$ indicates the c-th channel of I, $K_j(c)$ is the j-th kernel relative to the c-th input channel, b_j is a bias term, and $*$ is the cross-correlation operation. In total there are $C_{output} \times C_{input}$ different kernels whose elements, plus C_{output} bias terms, which are the parameters to be learned by the network.

One can see that when the kernel is comparatively smaller than the image, the number of parameters is significantly reduced when confronted with a fully connected neural network. However, these parameters are used much more efficiently considering the particular structure of images. By the nature of the cross-correlation operation, a convolutional layer only acts on the image locally, in a neighborhood that is as big as the kernel itself: thus a convolutional layer ignores the connection between very distant pixels; moreover, for every output channel there is only a single kernel that 'slides' through the image with the same parameters, thus the convolution with a kernel can be interpreted as a standalone local operation, and stacking multiple output channels can be an efficient way of computing many of these.

Usually the kernel height and width are chosen to be equal and referred to as the kernel *size*. We note that if we start with an input image of size (H, W), because of the cross-correlation operation the output feature map will have height and width reduced by $s - 1$ pixels, where s is the kernel size. To have an output feature map with the same size as the input, it's possible to pad the input image, i.e. add to the borders of the image a sufficient number of arbitrary valued pixels to offset the difference given by the cross-correlation. Kernel sizes in our case are usually taken to be an odd number so that the padding pixels can be evenly split to the borders of the image.

We also note that the convolution operation of an image I with a kernel K can be expressed as the matrix multiplication of I with a block circulant matrix that depends on the parameters of the kernel [9]; this means that a convolutional layer is still a linear operation, and thus the same rules for backpropagation of a feedforward neural network can apply. Moreover, because

of the linearity of the convolutional layer, one usually has to compose its application in a neural network with a non-linear activation function, to assure the NN can represent more complicated functions. Neural network architectures that employ multiple convolutional layers with activation functions are then called *convolutional networks*.

3.1.1 Residual Connections

Residual Connections were introduced in [17] as a method to ease the optimization and training of very deep neural networks by propagating information more efficiently during the feed forward phase. They can also be employed effectively in particular when the input and output of the neural network are not expected to differ substantially.

Suppose for simplicity we are working with a neural network f_θ and an input-output tuple (x, y) where x, y are of the same dimension; the output of the neural network is then given by $\tilde{y} = f_\theta(x)$, where f_θ wants to approximate a target function $y = f(x)$. This network can be modified into another network f'_θ such that the output is instead given by

$$\tilde{y} = f'_\theta(x) := f_\theta(x) + x : \tag{3.4}$$

where to the output of the original neural network we've only added element-wise the input x without implementing any additional learnable parameters. This modification lets the new network f'_θ learn the *residual function* $f_r(x) = f(x) - x = y - x$, and is particularly useful when the input-output tuples both do not differ much and have similar structure, so that it is reasonable to expect that the residual function, which is their difference, should be significantly simpler than the original target function.

In a deeper neural network, it's possible for residual connections to 'skip' more than one layer: for example, given two layers $g_{\theta_1},\ g_{\theta_2}$ of a neural network $g_\theta(x) = g_{\theta_2} \circ g_{\theta_1}(x)$, this can be modified into

$$g'_\theta(x) = x + g_{\theta_2} \circ g_{\theta_1}(x) \tag{3.5}$$

These are sometimes called *skip connections* and have been used increase the stability of convergence of complex and deep neural networks, especially in Transformers.

Residual connections are particularly useful in the task of Super Resolution and deblurring, as the input-output pairs have essentially the same structure; introducing connections throughout a network has shown to help the forward flow of information and increase performance [18] [19].

3.1.2 Pixel Shuffle and Unshuffle

The operation of Pixel Shuffle was introduced in [20] as an alternative to existent methods of sub-pixel convolution to upscale a LR image or feature map to a HR one.

Suppose we have a feature map $F(c, h, w)$ of size (r^2C, H, W); the Pixel shuffle operator

$\mathcal{PS}$ returns another feature map of size (C, rH, rW) following:

$$\mathcal{PS}(F)(c, h, w) = F(c + Cr \cdot h\%r + C \cdot w\%r,\ \lfloor h/r \rfloor,\ \lfloor w/r \rfloor) \tag{3.6}$$

where $x\%y$ indicates the remainder of the integer division of x by y. The parameter r is usually called the scale factor. Pixel Shuffle is implemented at the end of a Super Resolution network as an up-scaling operation of a low resolution feature map; using this operation makes it possible to perform convolutions on a feature map of smaller dimensions but with a high number of channels (r^2C) rather than a feature map with larger dimension (rH, rW).

The inverse operation, called Pixel Unshuffle, goes then from a feature map G of size (C, rH, rW) to one of size (r^2C, H, W):

$$\mathcal{PS}^{-1}(G)(k, n, m) = F(k\%C, nr + \lfloor k/(Cr) \rfloor, mr + \lfloor k/C \rfloor - r\lfloor k/(Cr) \rfloor) \tag{3.7}$$

3.1.3 Layer Normalization

Layer Normalization was introduced in [21] and is defined, for an input x, by

$$\begin{aligned} \hat{y}(x) &= \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \\ \mathrm{LN}(x) &= \gamma \cdot \hat{y}(x) + \beta \end{aligned} \tag{3.8}$$

where μ and σ^2 are the mean and variance of the features of the input x (the latter computed with the biased statistical estimator), which is then normalized element wise. γ and β are optional learnable parameters that define an affine transformation on the normalized input. ϵ is a small constant used for numerical stability.

Layer Normalization is usually employed multiple times through a neural network to avoid the possibility of very deep networks producing outputs of problematically small or big magnitude, which would interfere with the stability of learning in the backwards pass.

3.2 Swin Transformer

The Transformer architecture [22] is a very well known neural network backbone who has found great success in the field of Natural Language Processing. It introduced in particular the attention mechanism, which enabled Large Language Models to achieve greater results in various language processing tasks compared to the previously used Recurrent Neural Networks.

The Vision Transformer (ViT) [6] was one of the first successful adaptations of the Transformer architecture to the field of computer vision, applying the Transformer attention mechanism to non-overlapping input image patches; this architecture was only intended for image classification tasks, where it achieved a much better trade-off of accuracy prediction and cost

complexity compared to convolutional networks. Following, many adaptations and improvements of the ViT were introduced to produce architectures that could be implemented for tasks other than image classification; one of these is the Shifted-Window (Swin) Transformer [5]. The Swin Transformer Architecture differs from the original Transformer in two aspects:

1. How the input images are handled and embedded into the network.
2. How attention is computed.

The following is a description of the Swin Transformer architecture where in particular we highlight the above two points.

3.2.1 Patch Embeddings

We suppose we are working with input images of dimensions (C, H, W). The first step is to create from the input image a series of patches and embed them into *patch embeddings* to feed them to a Transformer Block.

We select a patch size s which we suppose divides the image width and height, and divide the input image in HW/s^2 patches of size (C, s, s). Each of this patches is 'flattened' into a vector of size Cs^2, and these vectors are multiplied by a learnable embedding projection matrix P to a chosen embedding dimension D, to obtain what are called *patch embeddings*, more commonly called *tokens*. The matrix of these has then size $(D, HW/s^2)$.

The operation above can be implemented as a single convolutional layer by setting the number of output channel to exactly the embedding dimension D, and the kernel size and stride both to the patch size s. This is more efficient than handling the single image patches and multiplying them with a single projection matrix.

As is done in the traditional Transformer, to the patch embeddings a learnable parameter called the *classification token* of size D is appended, to obtain a matrix of embeddings of size $(D, HW/s^2 + 1)$. To these, a learnable matrix of *positional embeddings* of the same size is added component wise. This is in contrast with the original transformer where the positional encoding where not learnable parameters, but where computed with a certain rule as to encode the relative position of the tokens; however, the authors of the Swim transformer noted that letting the positional embeddings be learnable, performance increased at little cost.

The resulting patch embeddings are fed as input to a first Swin Transformer Block.

3.2.2 Swin Transformer Block

The Swin Transformer Block is a modification of the original Transformer only when it comes to the computation of the attention mechanism. If one were to use the original Transformer architecture, the attention would be computed between every token, which becomes inefficient as the number of tokens increases.

The authors of the Swin Transformer (like ViT) opted instead to compute attention locally, by partitioning the patch embeddings in non-overlapping *windows* each of a certain size w, and attention is computed only between tokens that belong to the same window. However, computing attention only within each window loses the global character of the original attention mechanism, because no relation is computed in patches that belong to different windows. To make up for this, attention is also computed within the original windows shifted in position by $w/2$, half of their size, so now parts of the patch embeddings that belonged to different window are found to be in the same. As the authors point out, this creates more windows, some of which are actually smaller than the chosen than the chose size w.

To remedy this, the authors of the Swin Transformer opt to shift the smaller windows in a cyclic manner as to combine them together to obtain windows of the same size and same number as the non-shifted case. However, it's not possible to straightforwardly compute attention on these, because the combined windows contain patches that were not adjacent in the original disposition; the attention is then computed on a masked input of patches in such a way that the mask only permits attention computation on the shifted windows. An illustration of this mechanism can be found in the original paper [5].

With this modification, the structure of a Swim transformer block is identical to the one of the original, with the attention layer succeeded by a fully connected linear layer, both of which are preceded by Layer Normalization; moreover, two residual connections are used to let the input skip these two layers. These transformers blocks are organized in pairs of two, where the first block computes local attention in the defined windows, and the second one computes attention in the shifted windows; these can then be stacked sequentially to obtain a deeper network. Given an input $\boldsymbol{z}^{l-1}$ to the l-th Swim transformer block, the order of forward computation is [5]:

$$\begin{aligned} \hat{\boldsymbol{z}}^{l} &= \text{W-MSA}(\text{LN}(\boldsymbol{z}^{l-1})) + \boldsymbol{z}^{l-1} \\ \boldsymbol{z}^{l} &= \text{MLP}(\text{LN}(\hat{\boldsymbol{z}}^{l})) + \hat{\boldsymbol{z}}^{l} \\ \hat{\boldsymbol{z}}^{l+1} &= \text{SW-MSA}(\text{LN}(\boldsymbol{z}^{l})) + \boldsymbol{z}^{l} \\ \boldsymbol{z}^{l+1} &= \text{MLP}(\text{LN}(\hat{\boldsymbol{z}}^{l+1})) + \hat{\boldsymbol{z}}^{l+1} \end{aligned} \tag{3.9}$$

where W-MSA and SW-MSA are respectively the Windows and Shifted Windows Attention Mechanisms (computed over multiple heads, see the following subsection), MLP a Multi Layer Perceptron (a feed forward linear network), and LN the Layer Normalization. The output of this transformer block pair is then $\boldsymbol{z}^{l+1}$.

3.2.3 Attention Mechanism

We report here the precise computation of the local window attention in the Swim Transformer.

As described before, the patch embeddings are divided into local windows of a certain size w, so a window has dimensions $D \times w^2$, where D is the embedding dimension. From these

we compute the *queries*, *keys*, and *values* matrices by linearly projecting them with learnable parameter matrices respectively $W_Q,\ W_K,\ W_V$, to another chosen embedding dimension q. These parameter matrices are the same across all windows. Indicating the input of the attention mechanism by Z, attention is then computed as follows:

$$\begin{gathered} Q = W_Q \cdot Z, \quad K = W_K \cdot Z, \quad V = W_V \cdot Z \\ \text{Attention}(Q, K, V) = \text{SoftMax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} + B\right) \cdot V \\ \text{where } \ \text{SoftMax}(x)_i := \frac{e^{x_i}}{\sum_j e^{x_j}} \end{gathered} \tag{3.10}$$

Differently from the original transformer, a learnable *relative position bias* B is added in the attention computation, which the authors found to significantly increase performance. After attention is computed for all windows, the outputs are merged together to the original input shape. In practice, the attention computation is repeated in a multiple number of *heads* that do not share parameters, and the results are concatenated with each other to give the final output. This is called Multi-Head Attention (MSA), and it lets the architecture learn different attention representation of its input at essentially no additional cost, if these computations are done in parallel.

3.3 Loss functions

Given two images $I_1(C, H, W)$, $I_2(C, H, W)$ of the same size, the loss function that the super resolution network minimizes should be a measure of similarity between the two as explained in the first Chapter. For simplicity, we display the loss functions as having two images as inputs; during training, the loss function is computed as the average over a mini-batch.

3.3.1 Pixel L_2 and L_1 losses

The most straightforward way to compare the similarity of the two images is the mean square error of the pixel values, also called the L_2 loss:

$$\text{MSE}(I_1, I_2) = \frac{1}{CHW} \sum_{k=1}^{C} \sum_{i=1}^{H} \sum_{j=1}^{W} \left(I_1(k, i, j) - I_2(k, i, j)\right)^2 \tag{3.11}$$

Another similar loss is the mean absolute error of the pixel values, or L_1 loss function:

$$\text{MAE}(I_1, I_2) = \frac{1}{CHW} \sum_{k=1}^{C} \sum_{i=1}^{H} \sum_{j=1}^{W} \left|I_1(k, i, j) - I_2(k, i, j)\right| \tag{3.12}$$

The Huber Loss is a variation that combines the two above to give instead:

$$\begin{aligned} \text{Huber}(I_1, I_2) &= \frac{1}{CHW} \sum_{k=1}^{C} \sum_{i=1}^{H} \sum_{j=1}^{W} \mathbb{H}(I_1(k,i,j),\ I_2(k,i,j)) \\ \text{where } \mathbb{H}(x,y) &= \begin{cases} (x-y)^2/2, & \text{if } |x-y| \leq \delta. \\ \delta(|x-y| - \delta/2), & \text{otherwise.} \end{cases} \end{aligned} \tag{3.13}$$

where δ is a chose parameter, usually 1.

3.3.2 Perceptual Losses

The aforementioned L_1 and L_2 losses and their variations are easy to implement, however when confronted with real world images that contain both global and local features rich in detail, empirically it has been found that they struggle to capture appropriately the perceptual differences between images, as human sight is able to.

To remedy this, a different set of loss functions has been introduced in the literature, which are called *perceptual* losses [23], or content losses. The aim of these is to consider the similarity of images using a 'deeper' representation of them, which should take into account the detail and structure that cannot be extrapolated from the bare pixel values differences.

To do this, one takes a pre-trained neural network image classifier, whose feature maps should intuitively represent a deeper structure present in the image, and computes a chosen pixel-wise loss between selected feature maps of two images I_1, I_2, keeping the weights of the pre-trained classifier always fixed. Explicitly, consider a neural network image classifier V that is made of the composition of a series of layers g_i: $V(I) = g_N \circ \cdots \circ g_1(I)$; one then chooses to 'truncate' the network at a certain layer i^*, defining an output feature map

$$V^{i^*}(I) := g_{i^*} \circ \cdots \circ g_1(I) \tag{3.14}$$

and defines the perceptual loss to be the pixel loss between the feature maps of the images (using e.g. MSE):

$$\text{PerceptualLoss}(I_1, I_2) = \text{MSE}(V^{i^*}(I_1), V^{i^*}(I_2)) \tag{3.15}$$

Using this type of loss has been shown to produce in some cases better results that the MSE or MAE alone. Note that in the network classifier V one expects shallow feature maps to be still similar to the input image, while deeper feature maps are supposed to represent more complex structures, so one can generalize the above definition to include multiple chosen layers $i_1^*, \ldots, i_k^*$ and their respective feature maps:

$$\text{PerceptualLoss}(I_1, I_2) = \text{MSE}(V^{i_1^*}(I_1), V^{i_1^*}(I_2)) + \cdots + \text{MSE}(V^{i_k^*}(I_1), V^{i_k^*}(I_2)) \tag{3.16}$$

and possibly weigh each different term depending on the aim of the task.

The perceptual loss can be used as a standalone loss or can be an used as an addiction the aforementioned pixel losses. In the latter case one needs to carefully weight the perceptual loss as its value is dependent on the architecture and layers chosen from V. A common backbone architecture chosen for V in perceptual losses is VGG [24].

3.4 Validation Metrics

For validating the performance of our networks for SR on a validation set, it's possible to use the same loss functions as above. Nonetheless, in the literature it is common to use other metrics, mostly PSNR and SSIM, which we now introduce.

3.4.1 Peak signal-to-noise ratio

Peak signal to noise ration (PSNR) is a common evaluation metric for the comparison of two images I_1, I_2 that is based on the MSE. It is defined in decibels as

$$\mathrm{PSNR}(I_1, I_2) = 10 \log_{10} \left(\frac{\mathrm{MAX}^2}{\mathrm{MSE}(I_1, I_2)} \right) \tag{3.17}$$

where MAX indicates the maximum possible pixel value admitted by the image range. A higher PSNR indicates a lower MSE and thus a higher similarity of images.

While PSNR is straightforward to compute and its interpretation is clear, it has been criticized for not being capable of encoding the visual differences that are humanly perceivable [23], as it inherits the disadvantages of MSE explained above.

3.4.2 SSIM

The *Structural Similarity* (SIMM) index was introduced in [25] as a method of image quality assessment that could correlate better to the human perceived quality than measures like PSNR. Given a single channel images $I_1(H, W), I_2(H, W)$ of the same size, SSIM is defined to be

$$\mathrm{SSIM}(I_1, I_2) = \frac{(2\mu_1\mu_2 + C_1)(2\sigma_{1,2} + C_2)}{(\mu_1^2 + \mu_2^2 + C_1)(\sigma_1^2 + \sigma_2^2 + C_2)} \tag{3.18}$$

where μ_i, σ_i are the mean and variance of image I_i, and $\sigma_{i,j}$ is the covariance of the two images.

$$\begin{aligned}\mu_i &= \frac{1}{HW}\sum_{h,w} I_i(h,w) \\ \sigma_i^2 &= \frac{1}{HW-1}\sum_{h,w}(I_i(h,w)-\mu_i)^2 \\ \sigma_{i,j} &= \frac{1}{HW-1}\sum_{h,w}(I_i(h,w)-\mu_i)(I_j(h,w)-\mu_j)\end{aligned} \tag{3.19}$$

and C_1, C_2 are small constants used for numerical stability. For images with multiple channels, SIMM is computed for every channel and the results are then averaged.

The structure of SSIM was also chosen to satisfy these two proprieties:

$$\begin{aligned}\mathrm{SSIM}(I_1, I_2) &\leq 1 \\ \mathrm{SSIM}(I_1, I_2) = 1 &\iff I_1 = I_2\end{aligned} \tag{3.20}$$

Both of which come from the following inequalities:

$$\begin{aligned}2\mu_i\mu_j &\leq \mu_i^2 + \mu_j^2 \\ 2\sigma_{i,j} &\leq \sigma_i^2 + \sigma_j^2\end{aligned} \tag{3.21}$$

which are clear. These also explain how the SIMM score relates to the images: the closer the mean and the variance of the two images are, the higher and closer to 1 their SIMM is.

In pratice, SSIM is not applied to the whole two images, but it's taken over a chosen amount of local sub-windows of certain size $s \times s$ to account for local features, and the computed SSIM scores are then averaged together to obtain a single score for the comparison of the two images. This is referred to as MSSIM.

3.4.3 ILNIQE

One of the problems of our particular task was the absence of a proper validation set, i.e. we did not have the possibility of validating the performance of our networks with the metrics above as the LR images we had to super resolve did not have a proper HR counterpart. As an attempt at a solution, we decided to use a *no-reference* (or blind) image quality assessment metric. The objective of these metrics is to evaluate the perceptual quality of an image *without* comparing it to a reference counterpart. The particular metric that we used, IL-NIQE [26], does this by fitting a multi-variate Gaussian (MVG) model on certain Natural Scene Statistics (NSS), which have been shown to adeptly represent image quality, on a varied dataset of HR images; when evaluating the quality of an LR image with no references, the same NSS are computed and fitted with a MVG model, and the metric is computed as a distance between the distributions

of the MVG fitted on the HR images and the MVG fitted on the LR image. As a result, a lower IL-NIQE score should indicate a better perceptual quality.

3.5 Adversarial Training

Generative Adversarial Networks were introduced in [7] as a new way to train and optimize generative neural networks. The procedure consists in pairing a generative network G_θ (the *generator*), whose objective is, given a certain input x, to generate an output y with a certain distribution p_y, with another neural network D_θ called the *discriminator*, whose objective instead is to distinguish if a certain sample comes from the real distribution p_y or if it was instead generated by the network G_θ. The discriminator is thus a binary classifier whose output is a number in $[0, 1]$ indicating the probability that its input comes from the real distribution p_y.

The generator G_θ and discriminator D_θ are trained concurrently so that the discriminator maximizes its chance of correctly distinguishing between real and generated samples, while the generator minimizes the probability that the output it produce is correctly classified by the discriminator. The related optimization problem given by the authors is

$$\min_{G_\theta} \max_{D_\theta} \mathbb{E}_y \left[\log(D_\theta(y))\right] + \mathbb{E}_x \left[\log(1 - D_\theta(G_\theta(x)))\right] \tag{3.22}$$

In practice this min-max optimization problem can be approached by sequentially solving the minimum problem relative to the generator and the maximum relative to the generator via stochastic gradient descent. In our case, for an input y_i of the discriminator we consider the cross entropy loss

$$BCE = -\left(z_i \log(D_\theta(y_i)) + (1 - z_i) \log(1 - D_\theta(y_i))\right) \tag{3.23}$$

where z_i is the label corresponding to the input, i.e. 1 when y_i comes from the real distribution and 0 when it comes from the generator. The discriminator is then fed as input a sample from the real data distribution and a sample generated by G_θ in an alternate manner with the relative label, and its task is to minimize the above loss. To output a probability $y_i \in [0, 1]$, the last activation function of the discriminator is usually taken to be the *sigmoid* function:

$$sigmoid(x) = \frac{e^x}{1 + e^x} \tag{3.24}$$

Since the generator in our case also has to minimize its own loss function L, we add the adversarial loss to it and weight it by a certain factor w. The generator's task is to make its output so the discriminator classifies it with label 1, so that we should add the binary cross entropy loss with $z_i = 1$:

$$\hat{L}(G_\theta(x_i), y_i) = L(G_\theta(x_i), y_i) - w \log(D_\theta(G_\theta(x_i))) \tag{3.25}$$

During the generator's update, the parameter of the discriminator are kept fixed.

Using adversarial networks intuitively should let the generator learn to compute outputs that better resemble the original distribution p_y. However, the use of a specialized network comes both with a computational burden, as there is another whole network to train, and problems of stability, as GANs have been found empirically to be difficult to train and converge. If the discriminator is not good enough to meaningfully distinguish its inputs, then also the feedback that the generator gets from the adversarial loss becomes useless. Ideally, the discriminator should start with a good edge on the generator, and as training proceeds the latter should start improving based on the adversarial loss, until the discriminator is unable to distinguish its output from a real sample.

3.5.1 Spectral Normalization

Spectral Normalization was introduced in [27] as a way of regularizing the learning and convergence of discriminators. In particular, the authors wanted to find a method to assure that the function that the discriminator represents is Lipschitz continuous, as a number of works (for example [7]) highlighted the importance of a bounded Lipschitz Norm in stabilizing discriminator training.

We remember that for a differentiable function $f : \mathbb{R}^n \to \mathbb{R}^m$, its Lipschitz norm is given by

$$||f||_L = \sup_{x \in \mathbb{R}^n} \sigma(\nabla f(x)) \tag{3.26}$$

where σ indicates the spectral norm of a matrix, which is its largest singular value. Moreover, for two differentiable function f, g, their composition's Lipschitz norm satisfies $||f \circ g||_L \leq ||f||_L ||g||_L$.

Given a linear neural network layer $g(x) = A \cdot x$, where A is the parameter matrix (for simplicity, written without bias term), its Lipschitz norm is thus given by

$$||g||_L = \sup_x \sigma(\nabla A \cdot x) = \sup_x \sigma(A) = \sigma(A) \tag{3.27}$$

Moreover, if we compose g with an element-wise applied activation function h, we can use the above inequality to write:

$$||h \circ g||_L \leq ||h||_L ||g||_L = ||h||_L \sigma(A) \tag{3.28}$$

By only employing activation functions h with Lipschitz constant less than unity (for example, RELU), the norm of the whole layer is then bounded above by $\sigma(A)$. Thus, it's possible to normalize the layer matrix by its spectral norm, to guarantee the Lipschitz constant of the whole Layer to be bounded by 1:

$$A_{SN} := A/\sigma(A) \tag{3.29}$$

One can then use a discriminator structured as a composition of linear operations (convolu-

tions included) all normalized like so, such that the whole discriminator's Lipschitz constant can be bounded by 1.

To precisely compute the spectral norm of a matrix one would need to derive its Singular Value Decomposition, which is computationally expensive to do for every training iteration. Therefore, the *power iteration method* is used to instead get an approximation of the biggest singular value; even at every iteration, the cost complexity of doing this is negligible compared to the training complexity of the discriminator.

3.6 Adam Optimization

Adam, introduced in [8], combines two algorithmic designs to improve on simple stochastic gradient descent, *momentum* and *root mean square propagation*.

The concept of momentum is used in stochastic optimization to accelerate and stabilize the convergence of the algorithm particularly in regions of small curvature of the function to be minimized; it defines a momentum variable m_t that keeps track of the previous gradient updates, and updates the parameters with an exponential average of these. Thus, instead of the stochastic gradient update rule $\theta_t \leftarrow \theta_{t-1} - \alpha_t g_t$, where $g_t = \nabla_\theta f(\theta)$ is the gradient of the function to be minimized, the parameters are updated as such:

$$\begin{aligned} m_t &\leftarrow \mu_t m_{t-1} + \alpha_t g_t \\ \theta_t &\leftarrow \theta_{t-1} - m_t \end{aligned} \tag{3.30}$$

Where μ_t is the exponential decay rate. Similarly, root mean square propagation keeps an exponential average with decay rate ν_t of the uncentered variance of the gradient $g_t^2 = (\nabla_\theta f(\theta))^2$ with a variable v_t, so that it can be used in the update rule to normalize the learning rate:

$$\begin{aligned} v_t &\leftarrow \nu_t v_{t-1} + (1 - \nu_t) g_t^2 \\ \theta_t &\leftarrow \theta_{t-1} - \frac{\alpha_t}{v_t} g_t \end{aligned} \tag{3.31}$$

Adam combines these two approaches by keeping track of exponential moving averages of the gradient and its variance by using two different decay parameters $\beta_1,\ \beta_2$. Moreover, since these averages are usually initialized to 0, there is a bias in their estimation especially in the first time steps; this is corrected by normalizing them at each time step. See Algorithm 3.1.

3.6.1 Nesterov Momentum for Adam

Nesterov Accelerated Momentum is a modification of the classical momentum update (3.30) that completes the momentum step before computing the gradient. This can be done in many

Algorithm 3.1 Adam for stochastic optimization.

Require: α_t: stepsizes; β_1, β_2: decay rates for exponential averages of mean and variance of gradients.
while θ_t has not converged
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
{Exponential moving averages of gradient and variance.}
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$
{Initialization Bias Correction}
$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
{Update rule, ϵ is used for computational stability}
$\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot \hat{m}_t / (\hat{v}_t + \epsilon)$
end while

ways, like it was proposed in [28]

$$\begin{aligned} g_t &\leftarrow \nabla f(\theta_{t-1} - \mu_t m_{t-1}) \\ m_t &\leftarrow \mu_t m_{t-1} + \alpha_t g_t \\ \theta_t &\leftarrow \theta_{t-1} - m_t \end{aligned} \tag{3.32}$$

The author of [16] propose a modification of the above that permits implementing Nesterov momentum in Adam:

$$\begin{aligned} g_t &\leftarrow \nabla f(\theta_{t-1}) \\ m_t &\leftarrow \mu_t m_{t-1} + \alpha_t g_t \\ \theta_t &\leftarrow \theta_{t-1} - (\mu_{t+1} m_t + \alpha_t g_t) \end{aligned} \tag{3.33}$$

which differs from the above as the momentum step relative to $t + 1$ is applied in the t-th iteration, with $\mu_t m_{t-1}$ then substituted by $\mu_{t+1} m_t$. Writing the Adam update rule (including the bias initialization) in terms of m_t and g_t, taking into account the change in μ_t, one obtains

$$\theta_t \leftarrow \theta_{t-1} - \frac{\alpha_t}{(\hat{v}_t + \epsilon)} \cdot \left(\frac{\mu_t m_{t-1}}{1 - \prod_{i=1}^{t} \mu_i} + \frac{(1 - \mu_t) g_t}{1 - \prod_{i=1}^{t} \mu_i} \right) \tag{3.34}$$

and using the aforementioned substitution the authors introduced Nesterov momentum into Adam as in Algorithm 3.2.

Algorithm 3.2 Nadam: Adam with Nesterov momentum.

Require: α_t: stepsizes; $\{\mu_t\}_{t\geq 1}$ decay rate for gradient moment; β_1, β_2: decay rates for exponential averages of mean and variance of gradients.
while θ_t has not converged
$t \leftarrow t+1$
$g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1-\beta_1)\cdot g_t$
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1-\beta_2)\cdot g_t^2$
$\hat{m}_t \leftarrow (\mu_{t+1} m_t)/(1-\prod_{i=1}^{t}\mu_i) + ((1-\mu_t)g_t)/(1-\prod_{i=1}^{t}\mu_i)$
$\hat{v}_t \leftarrow v_t/(1-\beta_2^t)$
$\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot \hat{m}_t/(\hat{v}_t+\epsilon)$
end while

3.6.2 Decoupled Weight Decay

A common regularization method for machine learning tasks is to add a penalty to the loss function $f(\theta)$ composed of the square norm of the parameters

$$\tilde{f}(\theta) = f(\theta) + \lambda ||\theta||_2^2 \tag{3.35}$$

This is called L_2 regularization, with hyperparameter λ. When one minimizes this penalized loss function with simple stochastic gradient descent, the update rule then becomes

$$\theta_t \leftarrow \theta_{t-1} - \alpha_t(\nabla_\theta f(\theta) + \lambda\theta_{t-1}) = (1-\lambda\alpha_t)\theta_{t-1} - \alpha_t \nabla f(\theta) \tag{3.36}$$

A very similar regularization technique is *weight decay* with decay parameter λ', which consists of the update rule

$$\theta_t \leftarrow (1-\lambda')\theta_{t-1} - \alpha_t \nabla f(\theta) \tag{3.37}$$

These two update rules are identical for SGD if $\lambda' = \lambda\alpha_t$. However, for optimization algorithms like Adam, these two are not the same, as the update rule does not explicitly depend on the current gradient of the loss function, but on a exponential average including previous steps. As pointed out by the authors of [29], in this case L_2 regularization doesn't scale every parameter by the same weight, as the penalized gradient is scaled by the moving averages; the authors then modify Adam's update rule as to mimic the simple SGD case to obtain what they call *decoupled weight decay*, which regularized the parameters by the same factor:

$$\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot \left(\frac{\hat{m}_t}{\hat{v}_t+\epsilon} + \lambda\theta_{t-1}\right) \tag{3.38}$$

The same modification can be applied to the version of Adam with Nesterov Momentum.

3.6.3 Kaiming Weight Initialization

A critical choice to be made is how to initialize the parameters of the neural network at the start of training. A common choice is to sample each parameter independently from a distribution that is symmetric around 0, such as a centered Gaussian distribution $\mathcal{N}(0, \sigma^2)$ or a uniform distribution $\mathcal{U}(-a, a)$. However, as the network becomes wider or deeper, the application of many layers and their respective activation functions might lead the gradients to explode or vanish in magnitude, especially if the activation functions 'saturate' the gradients, so that the network doesn't converge to a good minimum. A good initialization of the parameters that takes into account the size of the network and the nature of the activation functions helps to prevent this. In [30] the authors derive an initialization (called *Kaiming initialization*) method for linear layers that use rectified linear units as activation functions (such as RELU), with the assumption that a network composed of them should not drastically change the magnitude of its inputs. We report it briefly here.

We suppose we are working with layers of the type (2.5), where it is assumed that the elements of W_i are independent from the same distribution that is symmetric around 0 and has mean 0, that also the elements of x_i are from the same distribution and independent, and finally that x and W_i are independent of each other. We also assume that the biases b_i are initialized to 0. With these, the variance of the output z_i is equal to

$$Var(z_i) = n_i Var(W_i x_i) = n_i Var(W_i)\mathbb{E}[x_i^2] \tag{3.39}$$

And in the case of a RELU activation function between linear layers, we have $x_i = \max(0, z_{i-1})$

$$\mathbb{E}[x_i^2] = \int_{\mathbb{R}} x_i^2 p(x_i) dx_i = \int_{\mathbb{R}\geq 0} z_{i-1}^2 p(z_{i-1}) dz_{i-1} = \frac{1}{2} Var(z_{i-1}) \tag{3.40}$$

So that one has

$$Var(z_i) = n_i Var(W_i)\frac{1}{2} Var(z_{i-1}) \tag{3.41}$$

The authors argue then a reasonable initialization should change the variance of the outputs z_1 drastically, so that the above expression should be a constant; with multiple layers, it's possible to force this expression to be equal to 1 such that for each layer we have

$$Var(W_i) = \frac{2}{n_i} \tag{3.42}$$

So usually one chooses to initialize the values of W_i with a Gaussian distribution of $\mathcal{N}(0, \sqrt{\frac{2}{n_i}})$, and the biases to 0. This derivation relies on the nature of the activation function RELU and can be generalized to other rectified activation functions of the type $h_i = \max(x, \alpha x)$, with $\alpha < 1$, in which case the right side of equation (3.42) becomes $2/(\alpha^2 + n)$.

4

Super Resolution and Applications

We start this chapter with a formal definition of the problem of Super Resolution. We then describe the particular images that we were tasked to super resolve and the challenges that are specific to them, following with a description of our approach. We describe the degradation pipeline we used for generating training pairs and the modification we applied for PANCAM images, together with some data augmentation techniques we used; we then explain the details of the architectures we employed and their hyperparameters. We conclude by discussing the results we obtained, highlighting the differences between architectures, and with some final remarks about possible improvements.

4.1 Problem description

The problem of Super Resolution (SR) consists in, given a low resolution image I_{LR} that we suppose has been obtained by downsampling or/and degrading a high resolution counterpart I_{HR}, to accurately reconstruct the high resolution image. In the literature, two types of SR are usually considered, *classical* and *real-world* SR, which differ in how the low resolution image is assumed to have been produced.

In the simplest case of classical SR, it is supposed that the LR image has been obtained by downsampling the HR one by a certain integer factor s called *scale*, which we suppose divides its resolution, such that we can write:

$$I_{LR}(c, h, w) = I_{HR}(c,\ s \cdot h,\ s \cdot w) \tag{4.1}$$

If then I_{HR} has size $sH \times sW$, I_{LR} will have size $H \times W$. This downsampling operation can be generalized to non-integer scales by interpolating the HR image; common methods for images are nearest-neighbor, bilinear, and bicubic interpolation. However, an undersampling this simple is unlikely when one deals with real optical systems, as many other possible sources

of degradation come into play, such as noise and blur. These fall under the degradations that Real-World SR attempts to reconstruct.

In Real-World SR, it is in fact supposed then that the LR image is the result of applying to the HR image an unknown degradation operator $\mathcal{D}$ that maps in general images of size $H \times W$ to images of size $H' \times W'$, with $H' \leq H, W' \leq W$:

$$I_{LR} = \mathcal{D}(I_{HR}) \tag{4.2}$$

The operator $\mathcal{D}$ might be understood as a complex composition of downsampling, noise addition, blur, and other operations that reduce the visual quality of the image. It is usually assumed that the downsampling is done by an integer scale s, so that we still have $H' = H/s, W' = W/s$.

Both classical and real-world SR are *ill posed* problems, in the sense that the operations of downsampling and the degradation operator $\mathcal{D}$ are not injective, and so their inverses are not well defined.

We note that how much information is lost with a downsampling operation is lost depends on the sampling factor s. The Nyquist-Shannon Sampling Theorem [31] states that if a signal is composed of no frequencies above f_s, then it can be completely reconstructed by sampling it with a frequency of at least $2f_s$; if this condition (called the Nyquist Criterion) is not met, then some frequencies of the original signal are lost with the operation of sampling, producing an effect called aliasing. Aliasing is thus a necessary condition for SR [2].

We here describe briefly the two most common degradations that the operator $\mathcal{D}$ is composed of, namely the operation of blurring and the addition of noise, and respectively the problems of deblurring and denoising.

Blur and Deblurring

Blurring is one of the most common degradations that are deal with in real-world super resolution. In the simplest case blur is modeled as the result of the convolution of an image I with a kernel K:

$$I_{blur} = I * K \tag{4.3}$$

Convolution by a kernel K is not an injective operation and so even in this case information is lost. Blur might be an intrinsic property of the optical system that captures an image, in which case the kernel that describes it is called a Point Spread Function (PSF), or it could originate from other sources such as defocus or motion. The problem of deblurring consists in recovering I from I_{blur}.

Noise and Denoising

Another common degradation in real-world super resolution is the addition of noise, of which the most common type is *Gaussian additive* noise. Given an image I, the noisy counterpart

I_{noisy} is modeled to have been obtain by the addition of a noise vector N which we suppose has elements sampled independently from each other from a Gaussian distribution $\mathcal{N}(0, \sigma^2)$:

$$I_{noisy} = I + N \tag{4.4}$$

The problem of denoising consists again in recovering I from I_{blur}.

Neural Networks for Super Resolution and Image reconstruction

The problems of SR, Deblurring and Denoising of images can be tackled in various ways, but starting from [32], which introduced in particular a convolutional architecture for single-image SR which surpassed previous machine-learning methods, deep neural networks have consistently achieved state of the art results. Another important contribution came from the authors of [33], which first successfully implemented an adversarial training procedure for more realistic SR results. After the introduction of SwinIR [34], following works focused on Transformer architectures and variants, that currently achieve the best performance metrics.

Using the notation of the previous chapters, and given equation 4.2, the function that our neural networks want to learn is the *inverse* of the degradation operator, which takes as input a LR image and outputs the corresponding HR one:

$$\mathcal{D}^{-1}(I_{LR}) = I_{HR} \tag{4.5}$$

which in the simplest case consists in reversing a simple downsampling operation, and in the most complex reversing a composition of blur, noise, and downsampling. We repeat that the operator $\mathcal{D}$ is not injective, and so the above inverse is not well defined; however, one can suppose that for a given I_{LR}, many elements of its preimage $\mathcal{D}^{-1}(I_{LR}) = \{I_{HR} \,|\, \mathcal{D}(I_{HR}) = I_{LR}\}$ are not in fact *natural* but only byproducts of the non-injective nature of $\mathcal{D}$, and there is only one legitimate inverse that we are interested in recovering.

We also note that we focused on the problem of *single-image* SR, where we super-resolve based on the information of a single LR image input; in contrast, when multiple LR captures of the same subjects are available, the problem takes the name of *multi-frame* SR.

4.2 PANCAM Images

Our particular objective was to super-resolve and visually enhance images captured by a special hyper-panoramic lens system, which we refer to as PANCAM [3]. This panoramic lens captures single-channel images with a field of view (FOV) of $360°$ in the azimuth plane like many other omni-directional cameras such as fisheye lenses, however it is also able to view more than $100°$ in the zenith plane, with more than $40°$ below the horizontal plane in which the lens is positioned. As a result of this high FOV, the resulting images are heavily visually distorted and present singular degradations that in particular are *spatially variant*. To ease the processing of

these images, they are projected in an equirectangular plane according to the geometric model defined in [35], an example of which can be seen in 4.1.

Generally we denote these images as matrices I_{ij}, where then the i index refers to the zenith angle included in $[-\pi/2, \pi/2]$, and j refers to the azimuth in the interval $[-\pi, \pi]$. The images we worked with were of size 1800×3600 pixels.

The degradations in these images that are of interest to us are of two types:

1. The imaging system has a point spread function (PSF) that, convolved with the object captured in the image, produces blur. Particularly, when viewed in equirectangular projection, the blur spatially varies as a function of the zenith angle: in the image I_{ij} as i increases, so does the variance of the PSF and therefore the resulting blur is more pronounced. This makes it so the regions captured at 'low' zenith angle are much more distorted than regions at 'higher' angles. An example is in figure 4.2 Moreover, the variance of the PSF doesn't vary heavily as a function of the azimuth angle.

2. The process of projecting the images into an equirectangular plane is composed of both down-sampling and over-sampling operations, which we suppose to be unknown. Again these vary as a function of the zenith angle, however in a more complicated manner. Nonetheless, the images do not seem to present a high level of aliasing.

Moreover, as is the case with any imaging system, the captures are subjected to noise.

Figure 4.1: Example image taken by PANCAM in equirectangular projection.

The PANCAM main objective is to be used in a proposed robot named DAEDALUS [36] whose task is the exploration of lunar lava tubes; as a result, the main images that it should capture are of rock-like formations, which are fairly irregular and complicated in nature.

4.3 Our approach

We have described above the particular images we were tasked to super-resolve. As explained, the degradations in these images are fairly complex and essentially unknown in nature, except

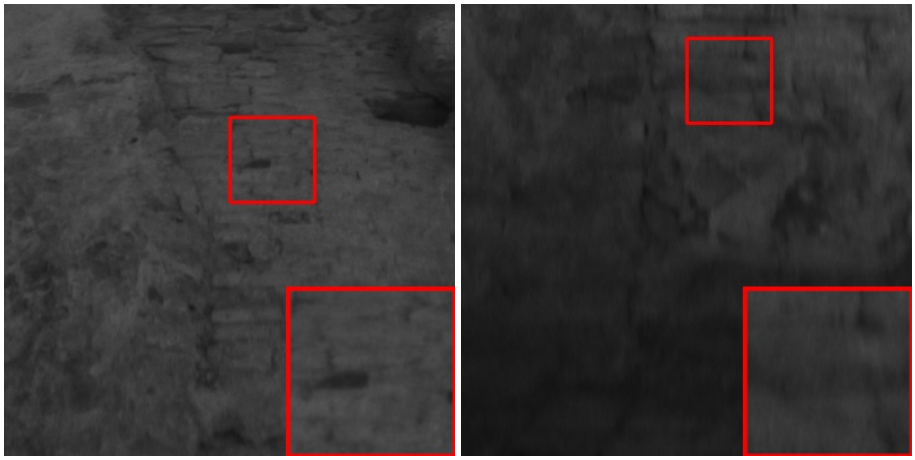


Figure 4.2: Degradation difference as a function of zenith angle in images taken by PANCAM. Both these image patches are taken at the same azimuth angle, but the left is at zenith angle of circa $\pi/3$ over the horizontal plane, while the right at $\pi/9$ *below* the same plane. It is clear that the PSF has higher variance in the latter image.

some inferences that can be made by directly inspecting the images themselves, which still can't be taken as accurate. For these reasons above, we decided to organize our work in tasks of increasing complexity, starting from simple classical SR, to Real-World SR with spatially invariant blur, and finally considering the complex degradations of PANCAM images.

Moreover, it was difficult during our work to produce a training dataset that was both complete and diverse enough to train a reasonably sized NN, especially considering the particular subject that the PANCAM images are supposed to capture, i.e. rock walls and formations. To train our NNs we then opted for a very generalized training by using publicly available datasets composed of a large amount of HR images representing a various range of subjects. This general approach's main disadvantage resides in the impossibility of achieving the best possible performance for a given task (in our case, the subject being rock formations captured by PANCAM) as this would require a task-specific training set; however, it does have its advantages. First, having a general-purpose model of SR opens the possibility of using it as a fine-tuning starting point for any particular sub-task, in the case a new specific dataset becomes available; secondly, a model fine-tuned with this approach not only takes then very little training resources in the fine-tuning stage, but is also usually able to surpass in performance a model trained *only* on the task-specific dataset.

There remains the problem of having a paired training dataset, i.e. generating the LR images from their HR ones. For the task of classical SR, it is simply sufficient to down-sample the HR images by any method and with the required factor. For Real-World SR and the PANCAM images this is not sufficient, there needs to be a way of degrading the images as to mimic the degradations that one wants to invert. For this, progress was made by the authors of ESRGAN [4] by introducing a 'degradation pipeline' that takes as input a HR image and degrades it

with a composition of blur, noise, and down-sampling of various nature to simulate a large distribution of real-world degradations. We used this pipeline for the Real-World SR task, and then we particularly modified it to also represent the specific degradations of PANCAM.

For classical SR, we only train a generator that super resolves the image, but for Real-World and PANCAM SR we also employ a discriminator architecture, which should encourage the generator to output more realistic images.

We organize the remaining sections as follows:

- We briefly describe the HR datasets that we used, and the degradations pipeline used to generate the LR counterparts, including some data augmentation techniques.
- We then describe the architectures we employed and their training parameters for each task of classical, Real-World, and PANCAM SR.
- We present our results, and we follow with an attempt at interpretation of the performance of our networks.
- We finish with some comments on possible improvements of our approach.

4.4 Dataset Description

As described above, we opted to train our networks using a general-purpose dataset. For this we used Nomos-v2 [37], a dataset composed of 6000 images of 512×512 resolution distilled from various datasets commonly used in the literature. For validation in the task of classical SR, we used the common Set5 [38], Set14 [38], and DIV2K bicubic downsampling [39] validation datasets. For validation of the Real-World SR task, we used the DIV2K unknown degradations validation set [39]. For validation on PANCAM images, we used the no reference metric ILNIQE, see section 3.4.3.

Dataset	Images	RGB Means	RGB Variances
NomosV2	6000	[0.3728 0.4235 0.4713]	[0.0689 0.0629 0.0748]
Set5	5	[0.3356 0.4420 0.5522]	[0.0699 0.0774 0.1042]
Set14	14	[0.3959 0.4503 0.5440]	[0.0690 0.0712 0.0708]
DIV2K Bicubic x2	100	[0.4067 0.4330 0.4311]	[0.0854 0.0717 0.0782]
DIV2K Unknown x2	100	[0.4069 0.4330 0.4311]	[0.0819 0.0678 0.0741]

The images are normalized within $[0, 1]$ by dividing them by the image pixel range; this makes it so the networks can be trained on 8-bit images but also possibly work on 16-bit images. We do not standardize the training images, as we have seen it doesn't seem to have any particular effect.

4.4.1 Synthetic Image Degradation

We describe here the procedure formulated by the authors of Real-ESRGAN [4] for generating a synthetic LR image I_{LR} starting from a high resolution image I_{HR} to generate a training pair (I_{LR}, I_{HR}).

1. First, the LR image is convoluted with a kernel K to model blur as in 4.3. The kernel K is chosen to be a certain pixel size k, that is always chosen to be odd; to generate the kernel, a grid of tuples G_{ij} of size $k \times k$ is generated following:

$$G_{ij} = (i - \lfloor k/2 \rfloor, j - \lfloor k/2 \rfloor) \tag{4.6}$$

Note that the center of the grid has value $(0, 0)$. Then, the probability density function of a Gaussian Bivariate distribution of zero mean and a chosen covariance matrix Σ is used as a model for the kernel point spread function by computing it at every point of the grid:

$$K'_{ij} = \frac{1}{2\pi\sqrt{det(\Sigma)}} e^{-\frac{1}{2} G_{ij} \Sigma^{-1} G_{ij}} \tag{4.7}$$

In practice Σ is chosen by selecting its two positive eigenvalues σ_1^2, σ_2^2 and applying a rotation matrix R as such:

$$\Sigma = R \begin{pmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{pmatrix} R^T \tag{4.8}$$

The kernel is then normalized so that its elements sum to 1:

$$K_{ij} = \frac{K'_{ij}}{\sum_{i,j} K'_{ij}} \tag{4.9}$$

and finally the image is blurred with a convolution by this kernel, $I^{blur} = I_{HR} * K$. To accommodate a possibly more variate distribution of point spread functions, the authors use a *generalized* Gaussian distribution to model the kernel, and also a bivariate 'plateau'-like distribution, which has PDF up to a normalization constant given by:

$$f(x) = \frac{1}{1 + x \cdot \Sigma^{-1} x} \tag{4.10}$$

Note that the the normalization constant is not needed as the kernel is normalized at the end.

However, the above process produces a spatially variant blur that is not representative of the degradations present in PANCAM images. To produce spatially variant, we decided to blur the HR image with two different kernels, K_1 and K_2 of respective variances σ_1^2, σ_2^2 with $\sigma_1^2 < \sigma_2^2$, to obtain two blurry images $I_1^{blur} = I_{HR} * K_1$, $I_2^{blur} = I_{HR} * K_2$

and then combine them as such.

$$(I^{blur})_{ij} = \left(1 - \frac{i}{H}\right)(I_1^{blur})_{ij} + \frac{i}{H}(I_2^{blur})_{ij} \tag{4.11}$$

where H is the height of the image. This is essentially a convex combination of the two blurry images that spatially depends on the height of the pixel i, and since convolution is a linear operation with respect to the kernel, the above is equivalent to convolving the HR image with a spatially variant kernel that varies as $\left(1 - \frac{i}{H}\right) K_1 + \frac{i}{H} K_2$. An example is in figure 4.3. Moreover, to better encode the spatially variant information within the image, similarly to the work of [40], we append a new channel to the LR images that mirrors equation 4.11 as such:

$$(C)_{ij} = \frac{i}{H} \tag{4.12}$$

So in the case of PANCAM images, the LR image input to our networks has 2 channels, the first one being the actual image, and the second one being this 'spatial encoding' channel.

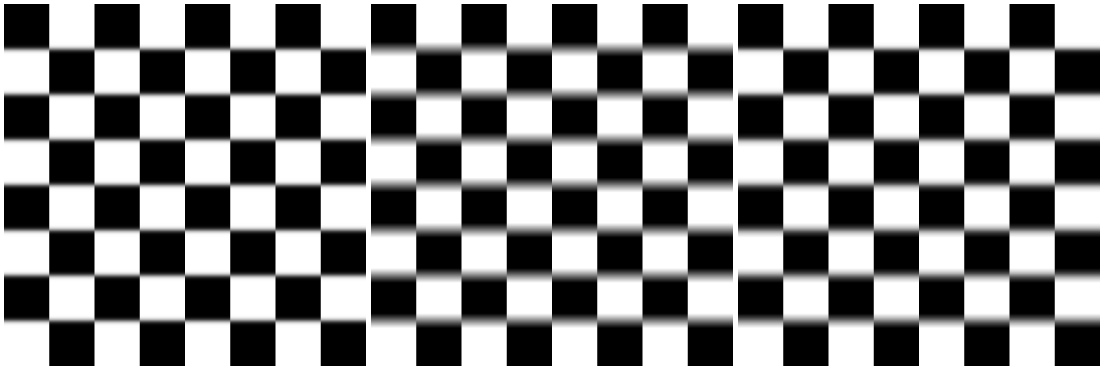


Figure 4.3: Illustration of our synthetic spatially variant blur on a check-board pattern. The right image is blurred with a kernel of variance $\sigma^2 = 3$, the center image with a kernel of variance $\sigma^2 = 12$, and the right is the combination of the two as formula 4.11, to obtain a linearly-variant blur degradation.

2. After the image is blurred, noise is added to the image as in 4.4. The authors consider two types of noise models: Gaussian, where to each pixel a 'noise' value sampled from a $\mathcal{N}(0, \sigma^2)$ is added; and Poisson noise (also shot-noise), which is instead modeled after a Poisson distribution; since the latter approaches the former by the central limit theorem, we only choose to apply Gaussian noise.

3. In the original Real-ESRGAN pipeline, the application of blur and then noise as above was repeated a second time to generate what the authors called *second order* degradations;

however, we found that applying these degradations twice generated images that were much more degraded than our purposes needed, especially as our networks are much smaller than the author's and thus do not have the same representation capabilities. To limit this problem but also not to limit the amount of degradations generated, the second order degradations are applied only with a small probability $p \ll 1$.

4. At the end of the above process, the image are down-sampled by the chosen SR scale s by a method between bilinear, bicubic, or adaptive average pooling.

We illustrate the degradation pipeline at each stage in figure 4.4.

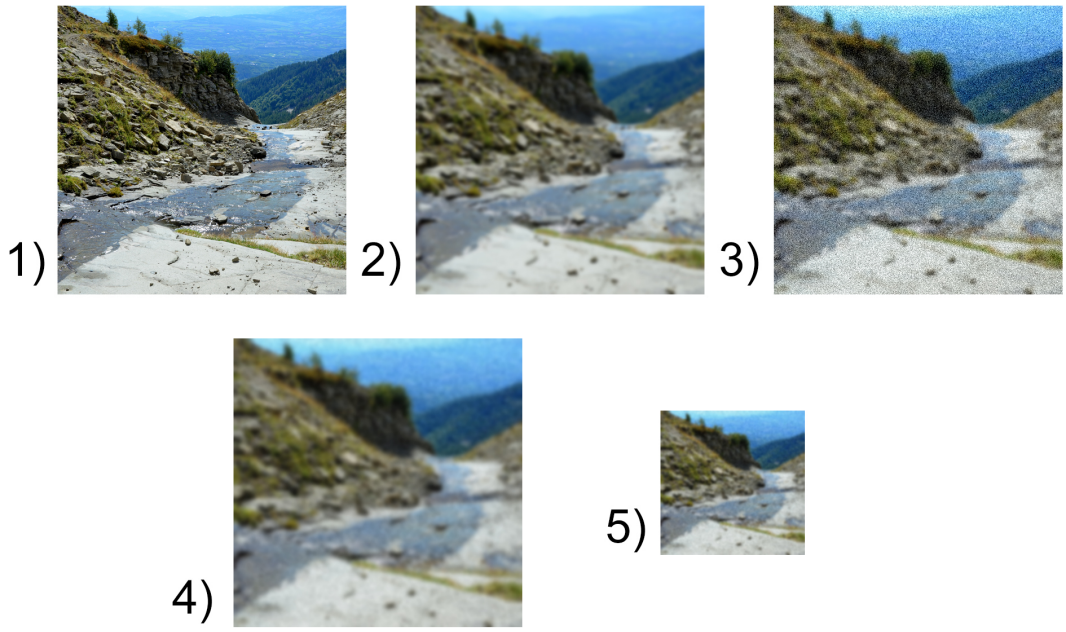


Figure 4.4: Illustration of the Real-ESRGAN Degradation Pipeline on a sample image: 1) Original HR image. 2) Blurred image. 3) Addition of Noise. 4) Second order Blur. 5) Final down-sampling. For this example we used a Gaussian Kernel with variances $\sigma_1^2 = 3$, $\sigma_2^2 = 5$, and the Gaussian noise variance set to 30.

Data Augmentation

As the number of parameters of a neural network increases, so does its complexity and expressiveness, but training it effectively to avoid overfitting requires bigger and more diverse training datasets, which are not always readily available. Moreover, bigger networks are in general more unstable during training and are more difficult to regularize.

Data augmentation is a set of techniques that focuses on improving the quality or diversity of the training data rather than improving on the training process itself; we report here four techniques that were introduced in the literature, 3 of which are data-agnostic, i.e. can be applied to any kind of input-output pairs, and one which was introduced specifically for the task of SR.

For simplicity we assume we are working with two input-output image pairs I^1_{LR}, I^1_{HR} and I^2_{LR}, I^2_{HR} in the training dataset, assumed to be the same respective sizes $H \times W$ and $sH \times sW$, where s is the scale of SR.

1. When dealing with image-like input, a straightforward augmentation technique is to rotate them by a multiple of $90°$ and/or flip them around an axis. In our case then both I_{LR} and I_{HR} can be rotated or flipped in the same manner to produce a new training pair.

2. **Mix-Up**[41] augments the training data by computing a new training sample (I'_{LR}, I'_{HR}) as a convex combination of the two above image pairs:

$$\begin{aligned} I'_{LR} &= \lambda I^1_{LR} + (1-\lambda) I^2_{LR} \\ I'_{HR} &= \lambda I^1_{HR} + (1-\lambda) I^2_{HR} \end{aligned} \tag{4.13}$$

   where $\lambda \in [0, 1]$. This parameter can be chosen each time at random to increase diversity. While Mix-Up empirically does increase the performance of general NNs, for images in particular it has the disadvantage of producing non-realistic captures.

3. **Cut-Mix** [42] similarly to mix-up, produces a convex combination of the two image pairs this time using a binary mask $M \in \{0, 1\}^{H \times W}$:

$$\begin{aligned} I'_{LR} &= M \odot I^1_{LR} + (1-M) \odot I^2_{LR} \\ I'_{HR} &= M \odot I^1_{HR} + (1-M) \odot I^2_{HR} \end{aligned} \tag{4.14}$$

   where $\odot$ indicates the element-wise product. In particular the mask M is built from choosing a bounding box B which has top left corner (b_x, b_y) and bottom right corner $(b_x + l_x, b_y + l_y)$ chosen at random following

$$\begin{aligned} b_x \sim Unif(0, H-1) \quad b_y \sim Unif(0, W-1) \\ l_x = H\sqrt{1-\lambda} \quad l_y = W\sqrt{1-\lambda} \end{aligned} \tag{4.15}$$

   where λ is again in $[0, 1]$ and essentially controls the area ratio of B with respect to the whole image, which from the above is $1 - \lambda$. M is then defined to be 1 in coordinates inside of B, and 0 outside of it. Cut-Mix was introduced to improve the performance of image classifiers; for Super Resolution, while it does not produce completely unrealistic

image pairs like Mix-Up, the binary nature of the mask M produces a sudden change in the features of the image.

4. **Resize-Mix** [17] A bounding box B and a binary mask M following the same process of Cut-Mix are chosen. Then, I^2_{LR}, I^2_{HR} are downsampled to the same size (l_x, l_y) of the bounding box, and the augmented training pair is given again by the same formula as Cut-Mix. This has essentially the same disadvantages of Cut-Mix.

5. **Cut-Blur** [43] is a data-augmentation technique specifically formulated for the task of super resolution. Given a single training pair I_{LR}, I_{HR}, it generates a new low resolution image following:
$$I'_{LR} = M \odot I^s_{LR} + (1 - M) \odot I_{HR} \tag{4.16}$$
where I^s_{LR} is the original LR image interpolated by the SR factor s and M is chosen as in Cut-Mix. I'_{LR} is then interpolated back down by the same factor s, and the new training pair is (I'_{LR}, I_{HR}). Note that the same HR image is used, while the LR is modified to include part of the HR counterpart. Differently from the previous augmentation processes, Cut-Blur does not produce sudden changes of content by using different image pairs, and thus keeps the image consistent; moreover, since only parts of the LR image now need to be super-resolved or de-blurred, Cut-Blur should also enable NNs to learn *where* to super resolve.

While we have explained the above augmentations with two image pairs, in practices these are applied to a whole mini-batch at each training iteration.

The authors of Cut-Blur also empirically found that employing a various array of data augmentation techniques provided better performance in various tasks than using any single one of them; following this approach, we use in our SR task all of the above techniques, by choosing randomly one of them (or none) at each batch-iteration.

4.5 Architectures

We describe here the architectures we have employed in detail. For the generator networks, i.e. the ones that super-resolve the input images, we used an architecture based on convolutional layers and one based on the Swin transformer. For Real-World SR and PANCAM images, the generators are trained concurrently with a discriminator, which is again convolutional in nature. We suppose in generality that the networks take as input an LR image patch I_{LR} of size $s \times s$ with c channels.

4.5.1 Convolutional Architecture

The convolutional architecture is based on RRDBNet [19]. It starts with a Pixel Unshuffle $\mathcal{PU}$ Layer of factor 2, so that the input image patch becomes of size $s/2 \times s/2 \times 4c$. Then, a first

convolutional layer A_1 outputs a feature map with a definite C channels; this convolutional layer has a kernel size of 3 and pads the image by 1 constant pixel to mantain its spatial size of $s/2 \times s/2$.

$$z_1 = A_1(\mathcal{PU}(I_{LR})) \tag{4.17}$$

The feature map z_1 then goes through a chosen number N of 'Residual in Residual Dense' Blocks (RRDB) which can be described as

$$z_{j+1} = \mathrm{RRDB}(z_j) = \alpha \cdot \mathrm{RDB} \circ \mathrm{RDB} \circ \mathrm{RDB}(z_j) + z_j \tag{4.18}$$

where each of the three 'Residual Dense' Blocks (RDB) is composed of 5 convolutional layers $B_i, i = 1, ..., 5$ with kernel size 3 and padding 1, that produce feature maps of an increasing number of channels as described by the following rule

$$B_i : C + (i-1)D \to D \tag{4.19}$$

where D is another chosen number of features that the authors of RRDB call the 'growth' of the feature map. Following the increasing number of channels, the subsequent outputs of each layer B_i (that we call w_i) are concatenated in the channel dimension with the output of the next layer as a form of residual connections; this is described by the following:

$$\begin{gathered} w_1 = h(B_1(z_j)) \\ w_{i+1} = h(B_2(w_i) \oplus w_i) \quad \text{for } i = 1, 2, 3, 4 \\ \mathrm{RDB}(z_j) = \alpha w_5 + z_j \end{gathered} \tag{4.20}$$

where by $\oplus$ we denote channel-wise concatenation, and h is the *leaky-RELU* activation function, applied element-wise: $\mathrm{leakyRELU}(x) = \max(x, 0.2x)$.

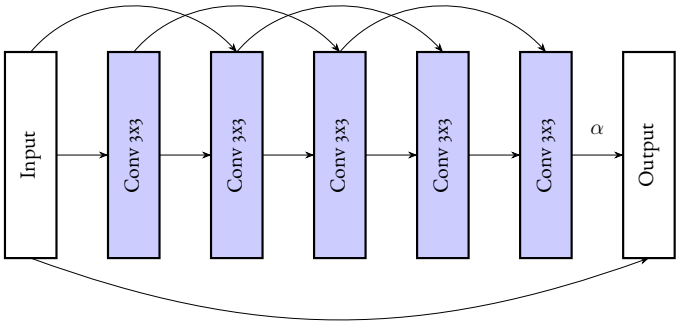


Figure 4.5: Illustration of a Residual Dense Block as in formula 4.20.

The output of the N-th RRDB $z_{N+1}$is then passed through another convolutional layer A_2

as above, and a residual connection is used with the initial feature map:

$$r = A_2(z_{N+1}) + z_1 \tag{4.21}$$

Finally the feature map is upsampled two times by nearest-mode interpolation of factor 2 US^2 and passed through another two convolutional layers A_3, A_4 like the previous with activation function h as above. The output of the network I_{HR} is given by a two final convolutional layers A_5, A_6, that respectively return a feature map with C features and finally the required number of output channels.

$$I_{HR} = A_6 \circ A_5^h \circ A_4^h \circ \mathrm{US}^2 \circ A_3^h \circ \mathrm{US}^2(r) \tag{4.22}$$

We note that in equation 4.18 and 4.20, the parameter alpha was set by the authors of RRDB net to be equal to 0.2; we instead let this be another learnable parameter. The parameters of the convolutional layers with leaky-RELU are initialized with Kaiming initialization 3.6.3 with the proper constant.

Parameter Free Attention

We augment the previous CNN based architecture following the works of [44] by the attention module introduced in [45]. In particular, after each RRD Block, the *energy* E_{hk} of each feature map z_j is computed, defined for each element $(z_j)_{hk}$

$$E_{hk}^j = \frac{4(\hat{\sigma}^2 + \lambda)}{((z_j)_{hk} - \hat{\mu}) + 2(\hat{\sigma}^2 + \lambda)} \tag{4.23}$$

where $\hat{\mu}, \hat{\sigma}^2$ are the mean and variance of the feature map channel where $(z_j)_{hk}$ is, and λ is a regularization parameter; the attention of $(z_j)_{hk}$ is given by $sigmoid(1/E_{hk}^j)$ and is multiplied element-wise with the input feature map:

$$Att(z_j) = z_j \odot sigmoid\left(\frac{1}{E^j}\right) \tag{4.24}$$

The output of each RRDB is then given by equation 4.18, where the attention is computed after the composition of the three RDBs. We verify empirically that this attention module improves the performance of our CNN architecture for the task of SR. As suggested by the authors, we choose $\lambda = 1e\text{-}4$.

4.5.2 Swin Transformer Architecture

The Swin Transformer Architecture is based on SwinIR [34], where we modify in particular the 'Residual Swin Transformer Block' (RSTB).

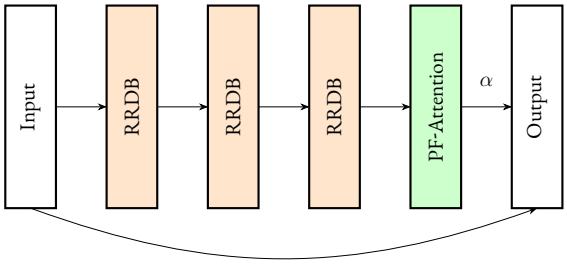


Figure 4.6: Illustration of a Residual in Residual Dense Block with Parameter-free Attention as in formula 4.18.

The network starts with a first convolutional layer A_1 with kernel size 3 that outputs a feature map with C channels (the embedding dimension), and this goes through a patch embedding layer $\mathcal{PE}$.

$$z_1 = \mathcal{PE} \circ A_1(I_{LR}) \tag{4.25}$$

The resulting feature map is passed through a number N of Swin Transformer Blocks that we denote by $B_i, i = 1, ..., N$. Here we modify the architecture proposed by the authors of SwinIR by following the same logic as RRDBNet [19] and [18] by introducing residual connections by means of concatenations of feature maps. Each RSTB B_i is composed of two *residual groups* R_i^1, R_i^2 that respectively take as input a feature map with C and $2C$ channels, and output a feature map of the same size and number of channels. In between these two, we insert a residual connection with the input of the block, and a subsequent convolutional layer A_i^1 that takes and outputs $2C$ features, followed by the leaky-RELU activation function h. After the second residual group, another convolutional layer A_i^2 that outputs C features (with no activation) is used. The whole action of a block B_i can be understood as

$$z_{j+i} = B_i(z_i) = A_i^2 \circ R_i^2 \circ h \circ A_i^1(R_i^1(z_i) \oplus z_i) \tag{4.26}$$

Where $\oplus$ again denotes concatenation in the channel dimension. We illustrate this in figure 4.7 Each residual group R_i is made of a number K of Swim Transformer Blocks 3.2.2 that as explained compute attention on local windows and their shifted counterpart. In each Swim Transformer the fully connected network is made up of two linear layers with GELU activation function, with $2C$ hidden units.

For simplicity of notation we do not write in formula 4.26 the fact that when passing from a residual group to a convolution one needs to *unembed* the image patches, and vice versa when one goes from a convolution to a residual group one needs to embed them.

After the N Residual Swin Blocks, the resulting feature map z_{N+1} is passed through another convolutional layer A_2 and a residual connection via element-wise addition is made with the

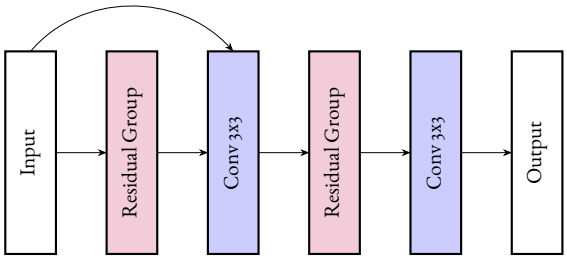


Figure 4.7: Illustration of a Residual Swin Transformer Block as in formula 4.26.

first feature map; another convolutional layer A_3 with leaky-RELU follows:

$$w = A_3\left(A_2(z_{N+1}) + z_1\right) \tag{4.27}$$

This last feature map is passed through a Pixel-Shuffle layer $\mathcal{PS}$ of required scale s and a last convolutional layer A_4:

$$I_{HR} = A_4 \circ \mathcal{PS}(w) \tag{4.28}$$

4.5.3 Discriminator Architecture

For the discriminator we considered an architecture similar to U-Net [46]. The network takes as input a HR image I_{HR} and outputs the probability $y \in [0, 1]$ that the image is not super resolved by the respective generator.

It starts with 3 subsequent convolutional layers A_1, A_2, A_3 followed by the leaky-RELU activation function h that terminate with a parameter-free attention 4.5.1 computation. The first layer A_1 outputs a feature map of embedding dimension C.

$$\begin{aligned} z_0 &= h(A_1(I_{HR})) \\ z_1 &= h(A_2(z_0)) \\ z_2 &= Att(h(A_3(z_1))) \end{aligned} \tag{4.29}$$

The above convolutional layers have a kernel size of 4 and a stride of 2, so the image is spatially downsampled by a factor of 2; moreover, each layer doubles the number of features, so z_2 has $4C$ channels. The feature map is then upsampled by bilinear interpolation of factor 2 US^2 and passed through another three convolutional layers A_4, A_5, A_6 of kernel size 3 and pad 1, enhanced by residual connections with the previous feature maps and a final parameter free attention computation:

$$\begin{aligned} z_3 &= h \circ A_4 \circ \mathrm{US}^2(z_2) + z_1 \\ z_4 &= h \circ A_5 \circ \mathrm{US}^2(z_3) + z_0 \\ z_5 &= Att(A_6(z_1)) \end{aligned} \tag{4.30}$$

The final layer A_6 outputs a 1-channel feature map which is passed element wise through a sigmoid function, and the final output is the average of the elements:

$$y = \text{mean}(sigmoid(z_5)) \tag{4.31}$$

Note that while we output a single probability $y \in [0, 1]$, z_5 has the same dimensions as the input I_{HR}, so the discriminator could be trained to output pixel-wise probabilities. Each of the above convolutional layers of the discriminator is normalized with Spectral Normalization as explained in section 3.5.1.

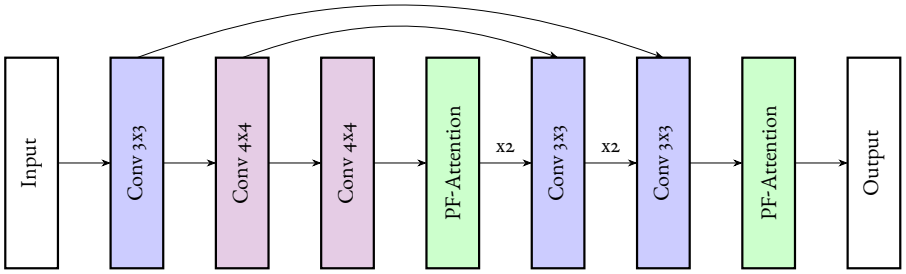


Figure 4.8: Illustration of the U-Net discriminator architecture.

4.5.4 Training Parameters

In table 4.1 we report the main architectural details such as the embedding dimensions and number of blocks for each architecture and each SR task. To give an idea of the model's complexity we also report the number of learnable parameters and the number of Multiply-Accumulate (MAC) operations, that is the number of operations of the type $a + b \cdot c$ for real numbers a, b, c in the forward pass for a given input size.

For the Perceptual Loss in all the tasks we used a pretrained VGG19 classificator network [24] and computed the $L1$ loss of the first 3 features maps (after the convolutional layers), with respective weights $\{0.1, 0.1, 1, 1\}$, as the authors of Real-ESRGAN did.

For all of our tasks we focused on a SR scale factor of 2. In the following sections we describe the hyperparameters used for our tasks, particularly the losses weights and degradation pipeline parameters. We note that these were mostly chosen heuristically.

For all the Transformer architectures, the window size is set at 8.

Classical SR

For Classical SR, we employ only the generator with the $L1$ pixel loss and perceptual loss with respective weights 1 and $1e$-4. We find that both the MSE and Huber loss are less performant compared to the $L1$ loss; however some authors [18] note that training an $L1$ model and sub-

sequently fine-tuning it with the MSE loss improves PSNR performance. We don't use a discriminator in this task as it seems to produce too many artifacts.

The HR training images do not go through any complex degradations but a simple downsampling operation using a bilinear, bicubic or average-pooling interpolation method to produce the LR counterparts.

For Validation we use Set5 [38], Set14 [47] and DIV2K Bicubic Downsample [39].

Real World SR

For Real World SR, the generator is trained on $L1$, perceptual and adversarial losses with weights 1, $1e$-4, $1e$-2.

For generating LR images we use the original degradation pipeline of Real-ESRGAN described above, where the blur is given by only Gaussian kernels of variance randomly selected within $[0, 1]$, and a Gaussian noise of variance within $[0, 0.1]$. This where chosen heuristically by inspecting the chosen validation set. The second order degradation is only applied with a probability of $p = 0.01$ per batch with kernel variance in between $[0, 0.25]$. We find that increasing in particular the variance of the Gaussian kernels enables the network to generate more visually pleasing images (mostly in sharpness) however at the cost of the validation metrics.

For validation we use DIV2K Unknown Downsample [39].

PANCAM Images

For PANCAM images we use the same losses and respective weights as Real World SR.

For generating LR images we use the degradation pipeline modified to give a spatially variant blur. First, the HR images are converted to a single channel by taking the average of the RGB values. We only use anisotropic kernels for this task, i.e. the covariance matrix of 4.7 is not scalar, mimicking the degradations of PANCAM images. Following our modified pipeline (see 4.11), we have to generate two kernels K_1, K_2 whose zenith variances are increasing as a function of the zenith angle.

To do so, a random zenith angle $\theta \in [-\pi, \pi]$ is chosen and normalized within $[0, 1]$. The azimuth variance σ_1^2 is always chosen within $[0.05, 5.0]$ for both kernels, while σ_2^2 for the first kernel is chosen as a function of θ in the interval $[(1+\theta)0.5, (1+\theta)8.0]$. We define Θ to be the zenith angle (normalized in $[0, 1]$) that the input HR image covers* , the variance of the second kernel is chosen in the interval $[(1 + \theta)8, (1 + \Theta)8]$; note that in this manner the variance of the second kernel is always bigger than the first, consistent with the degradations present in PANCAM images.

The LR images are blurred two times and combined for a variant blur as described in 4.11, and the respective supplementary channel 4.12 is generated as a function of θ. Gaussian noise of variance between $[.1, 3]$ is added, and again with small probability $p = 0.05$ the image goes

*In our case, the equirectangular images taken by PANCAM were 1800×3600 pixels. So for an input image of size $h \times w$, the zenith angle it covers is defined in radians to be $\pi \frac{h}{1800}$

Task	Architecture	Dimensions	Parameters	MACs
Classical SR	CNN	$C = 64, N = 4, D = 16$	1,651k	11,84 G
	Swin	$C = 30, N = 3, H = 3$	503k	9,66 G
Real-World SR	CNN	$C = 64, N = 6, D = 16$	2,398k	14,90 G
	Swin	$C = 36, N = 3, H = 3$	1,230k	23,17 G
	Discriminator	$C = 16$	64k	230 M
PANCAM SR	CNN	$C = 48, N = 6, D = 32$	3,449k	16,95 G
	Swin	$C = 42, N = 4, H = 6$	2,191k	40,9 G
	Discriminator	$C = 16$	64k	230 M

Table 4.1: Architectural details for each SR task. C is the embedding dimension, N is the number of respective network blocks (for CNNs and Swin), and D is the 'growth' parameter (for CNNs). For Swim, we indicate the number of heads in each Transformer by H. MACs are computed for an input image of pixel size 128×128.

through a second order degradation with spatially *invariant* blur of the same variances chosen for K_1 above, and noise variance between $[0, 1]$.

Validation is done on a small dataset of 6 image patches that are representative of a varied distribution of degradations present in PANCAM images, and IL-NIQE [26] is used as a no-reference metric.

Optimization and Implementation

All networks are trained with NAdam [16] with hyperparameters $\beta_1 = 0.98, \beta_2 = 0.99$ and decoupled weight decay with $\lambda = 1e\text{-}2$. We use in all cases a mini-batch size of 15. We train all networks for $100k$ mini-batch iterations, starting from a learning rate of $2e\text{-}4$ and halving it every $25k$ iterations. When training a generator and a discriminator, each network is trained for one mini-batch iteration sequentially. We note that we have tried similarly to the authors of Real-ESRGAN [4] to stabilize the training of our networks by using an exponential moving average of the learnable weights during training, however with no significant effect.

We implement our architectures and training procedure in Pytorch [15], using the packages of BasicSR [48] and NeoSR [37] as backbones. We train our networks using a single NVIDIA V100 GPU.

4.6 Results and Discussion

We report here the results in the three tasks we have organized as in section (4.3). The training and validation plots are generated through the Pytorch implementation of Tensorboard.

4.6.1 Classical SR

In figure 4.10 we plot the training and validation curves. Although training is not extremely stable, the Pixel and Perceptual losses steadily decrease throughout, while the validation metrics confirm the learning of our networks. As one can see the Swin-based architecture outperforms the convolutional one both with respect to the training loss and in the validation metrics, while having a significantly lower number of parameters. However, the convolutional one ends up being much faster both in training and evaluation. We report in table 4.2 the best PSNR and SSIM for both architectures achieved on the validation sets, although these are obtained at different iterations. Both architectures easily surpass the metrics obtained by bicubic interpolation.

Dataset	Bicubic	CNN	Swin
Set5	(32.055, 0.91283)	(32.84, 0.9246)	(**33.32**, **0.9296**)
Set14	(28.513, 0.8485)	(29.52, 0.8633)	(**29.87**, **0.8721**)
DIV2K Bicubic x2	(31.271, 0.8971)	(31.916, 0.9097)	(**32.3448**, **0.917**)

Table 4.2: PSNR and SSIM Results for Classical SR on the validation sets. The bicubic baseline is outperformed by both architectures, with the Swin Transformer obtaining the best resuls.

4.6.2 Real World SR

In figure 4.11 we plot the training and validation curves. As in the case of Classical SR, our networks are able to outperform bicubic interpolation on the validation set we chose (see table 4.3), although this time with less of a margin and also with less consistent improvements during training, as one can see from the validation curves. By manually inspecting the outputs we were able to understand the reason: while the networks are able to invert the degradations that the pipeline (4.4.1) generates, they struggle to generalize well to other types of degradations; in particular, all the blur kernels chosen in the degradation pipeline are essentially radially symmetric and have zero skewness, and it seems that the networks are not able to generalize beyond inverting the blur of this type of kernel. In fact, while it is common for PSFs and other types of blur sources to be radially symmetric, more complicated optical systems or blur sources (such as motion blur) are instead skewed.

Another source of problems is adversarial training. As one can see from the training curves, after $20k$ iterations or so the adversarial loss of the generator (and similarly, the discriminator loss) essentially stabilize. This is a common problem in GAN training, where after some iterations the discriminator is not able to distinguish meaningfully between real images and super resolved ones anymore, and its feedback to the generator becomes useless. This problem could be solved by more carefully tuning the loss parameters, or by finding a better equilibrium between the complexities of the generator and the discriminator.

Dataset	Bicubic	CNN	Swin
DIV2K Unknown x2	(25.119, 0.7046)	(25.47, 0.711)	(**25.54**, **0.7136**)

Table 4.3: PSNR and SSIM Results for Real-World SR on our validation dataset.

4.6.3 PANCAM SR

In figure (4.12) we report the training curves for both networks and respective discriminators.

Contrary to the architectures chosen for Real-World SR, one can see here from the training curves that there is a better equilibrium between the discriminator and generator, as the adversarial loss does not stabilize after some iterations but continues to steadily improve in favor of the generator. In fact, it is possible that our architectures could benefit from prolonged training iterations. From the validation curve one can see that both architecture easily achieve lower IL-NIQE scores compared to the 'baseline', which is the IL-NIQE score of the LR validation dataset. However, while the convolutional architecture slightly improves its scores during training, the Swin Transformer's metric gets worse over more iteration; manually inspecting the outputs does confirm that the Transformer architecture IL-NIQE scores are generally higher, but it's difficult to pinpoint exactly why. In figure 4.13 we highlight some SR examples taken from image 4.1, comparing some LR image patches to the SR outputs of our two architectures. As can be seen, we see notable improvements in image clarity which are confirmed by lower IL-NIQE scores. However, as the zenith angle gets lower and the blur becomes stronger, the networks struggle to super-resolve and deblur effectively, as in figure 4.14. While it is objectively harder to invert degradations that are stronger in magnitude, it is possible that our networks are not complex enough, or that we have not selected the correct hyperparameters in the degradation pipeline.

We also note that we have tried various combinations of losses and weights for PANCAM images. The main difference that we have seen is that while training an adversarial network does produce sharper images with lower IL-NIQE scores, it also tends to produce more artifacts and tends to smooth out complicated structures, e.g. rocks, while also seemingly amplifying noise. This highlights one of the shortcomings of IL-NIQE, i.e. the incapability of reflecting in its score the presence of artifacts of this genre. If one instead trains only a generator with e.g. a pixel loss, one obtains less sharp images with higher IL-NIQE scores, but with substantially less artifacts. We make a small comparison in 4.15.

In figure 4.9 we compare our classical SR, Real World SR, and PANCAM SR models in a sample image patch taken from 4.1. It can be seen that only our tailored PANCAM model is able to output a meaningful SR, confirmed by a lower IL-NIQE score.

4.6.4 Interpretation with Local Attribution Maps

For a more intuitive attempt at interpreting the behavior and performance of our networks, we make use of the work of [1] and compute the path-integrated gradients [49] of our networks

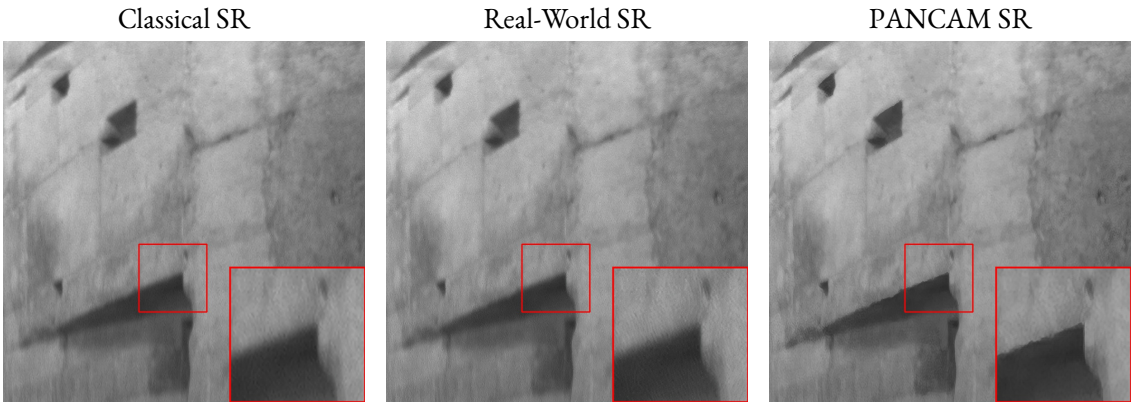


Figure 4.9: Comparison of our three SR tasks on PANCAM images, using our Swin architectures. Only our PANCAM model is able to super-resolve effectively.

output I_{HR} with respect to the input I_{LR}. These gradients can then be visualized and used as a measure of how much the features of the input I_{LR} influence the final results. Suppose that our network F takes $I_{LR} \in \mathbb{R}^{H \times W}$ as input and returns $I_{HR} = F(I_{LR})$; the authors of [1] defined a baseline path to be a path γ in LR image space parameterized by $\alpha \in \mathbb{R}^{\geq 0}$ and defined to be

$$\gamma(\alpha) = I_{LR} * K(\alpha) \tag{4.32}$$

where $K(\alpha)$ is a bivariate Gaussian kernel (as in 4.7) of covariance matrix $\alpha 1$, and $*$ is the operation of convolution; they then defined the local attribution map $LAM \in \mathbb{R}^{H \times W}$ to be the integral along the above path of the gradient of the output of F with respect to its input:

$$(LAM)_{ij} = \int \frac{\partial F(\gamma(\alpha))}{\partial \gamma(\alpha)_{ij}} \frac{\partial \gamma(\alpha)_{ij}}{\partial \alpha} d\alpha \tag{4.33}$$

The integration starts from $\alpha = 0$ and ends at a definite α^*, that we choose to be $\alpha = 1.2$. In practice, the gradient in not taken directly of $F(I_{LR})$, but first a feature-extraction operator D is applied to it so that the 'attribution' focuses on features; the authors of LAM choose D to be the gradient detector. Moreover, the gradients are computed only on a small image patch so that one can better understand if the model only takes advantage of local features or is able to take into account the global structure of the image. The integral above is approximated with discrete sum, in our case of 50 terms.

For visualization the LAM is first normalized within $[-1, 1]$ and then its absolute value is taken. Moreover, a Gaussian Kernel Density Estimator is fitted to the LAM distribution to better highlight its coverage. We also use the Diffusion Index (DI) indicator that the authors of LAM defined, as a measure of the amount of information that the network is able to take

advantage of.

We report a comparison based on LAM of our two PANCAM architectures in figure 4.16. It can be seen both from the visualization and the reported DI scores that the Swin-based architecture outperforms the CNN-based one, and is able to take advantage of more information when super resolving an image.

4.7 Possible Improvements

We list here some suggestions on how to improve our work.

We have highlighted that one of the shortcomings of the Real-ESRGAN degradation pipeline is the fact that all of its blur kernels have zero *skewness*. A simple improvement would be to add blur kernels with a distribution that has skew. A one-dimensional distribution that satisfies this is the *skew-normal* of parameter $\alpha \in \mathbb{R}$, which has PDF equal to

$$f(x) = 2\phi(x)\Phi(\alpha x) \tag{4.34}$$

where ϕ, Φ are respectively the PDF and CDF of a normal distribution. One can then add scale (variance) to this via an affine transformation. This distribution has been extended to the multivariate case [50], and thus it should be possible to generate a bivariate blur kernel from its PDF. Doing this introduces however more hyperparameters to be chosen, namely the two variances and skews in the bivariate case; moreover, as the variety of blur kernel increases, one would also need to increase the complexity of the respective SR network in order for it to perform well. Another topic we didn't focus on is hyperparameter selection, both for the parameters of the degradation pipeline and the loss weights; this could be a source of problems especially when training a model with a multiplicity of losses. As training a deep network is very resource intensive, selection of hyperparameters is even more difficult in this case. A partial solution would be to perform hyperparameter selection on a much smaller model, and using the selected hyperparameters to train a deeper model, however with no guarantee that the selected hyperparameters would be optimal. We also did not focus much on more complicated losses for image reconstruction, such as the focal frequency loss [51]. This loss in particular aims at improving the reconstruction and SR of images by considering their differences in the frequency domain, however we weren't able to include it in our training without causing significant instability in the first iterations. One solution could be to employ this loss only after a pre-training with the ordinary losses we have introduced. Another possible improvement would be to better make use of the 'spatial encoding' channel 4.12 that we employed inspired by [40]. The authors of this work use in particular a 'deformable' convolutional layer [52] with learnable offsets which takes as input the channel which encodes spatial information; the problem with this layer is that it is extremely slow to train compared to standard convolutions. To mitigate this problem, it might be possible to use our pretrained CNN and fine-tuning it by substituting the ordinary convolutional layers with the deformable ones.

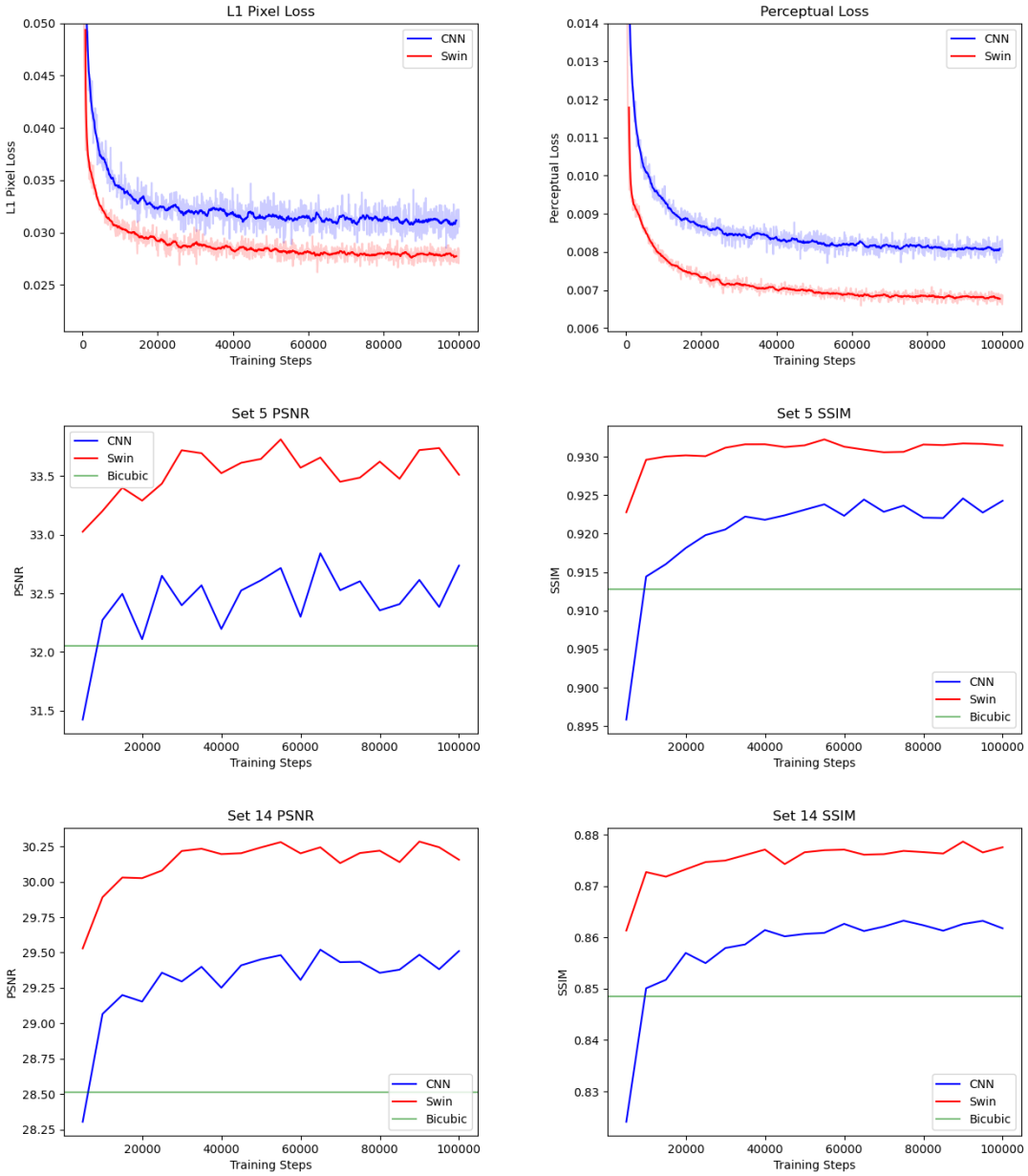


Figure 4.10: Classical SR: Training Loss graphs for L1 loss (top left) and Perceptual loss (top right). Following validation plots for Set5 and Set14 for the metrics of PSNR and SSIM, with bicubic baseline highlighted.

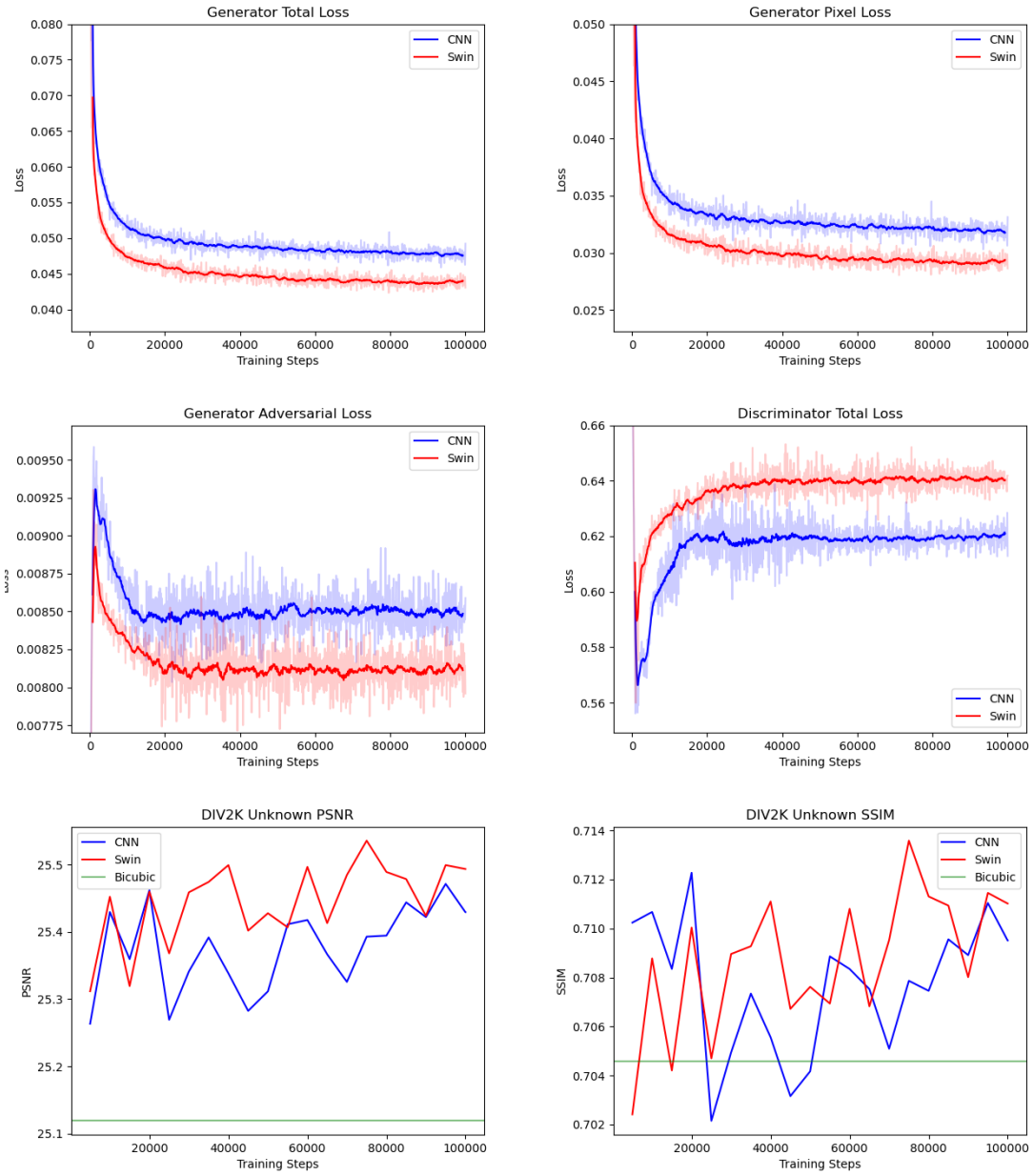


Figure 4.11: Real-World SR: Training Curves for the generator and discriminator. Following validation plots for the DIV2K Unknown Dataset, with bicubic baseline comparison.

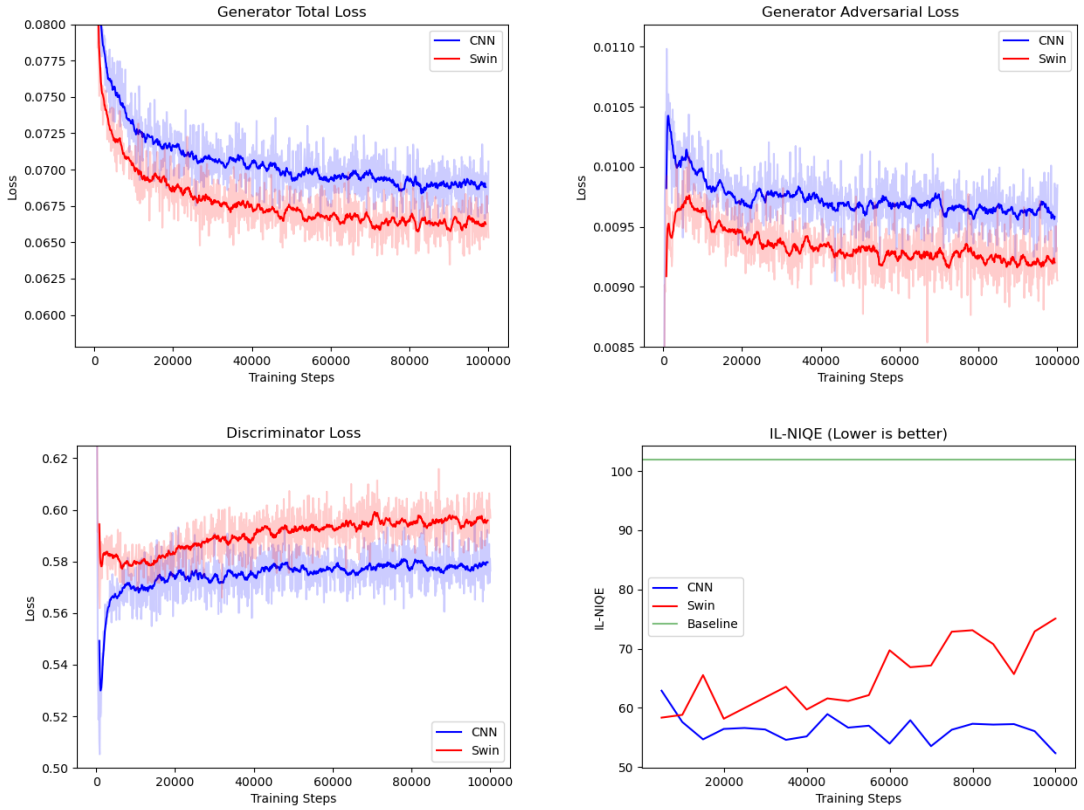


Figure 4.12: Training Loss Graphs for the Generator and Discriminator, PANCAM SR.

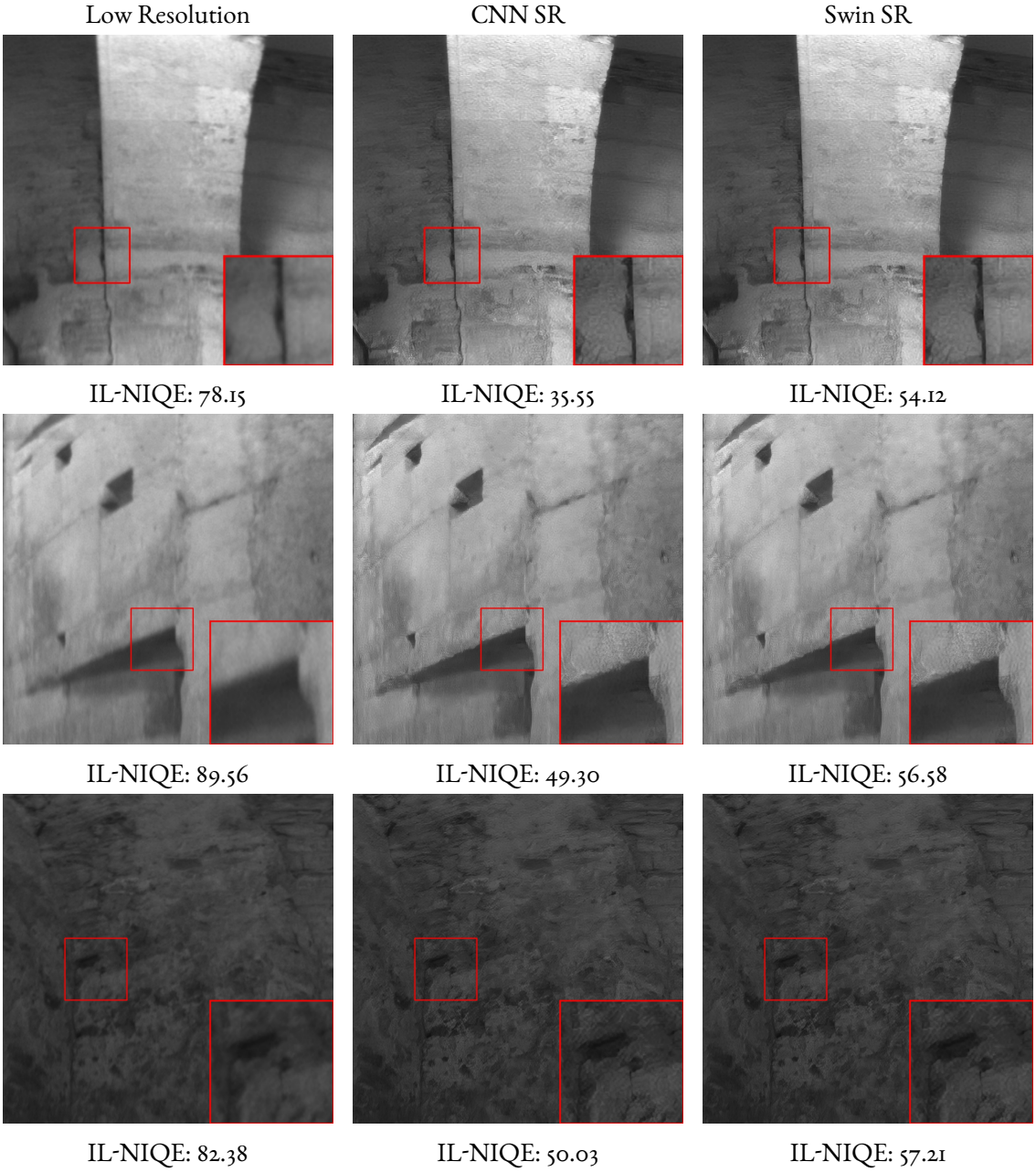


Figure 4.13: Example PANCAM image details and the SR images with our two architectures. Below each image we report its IL-NIQE metric.

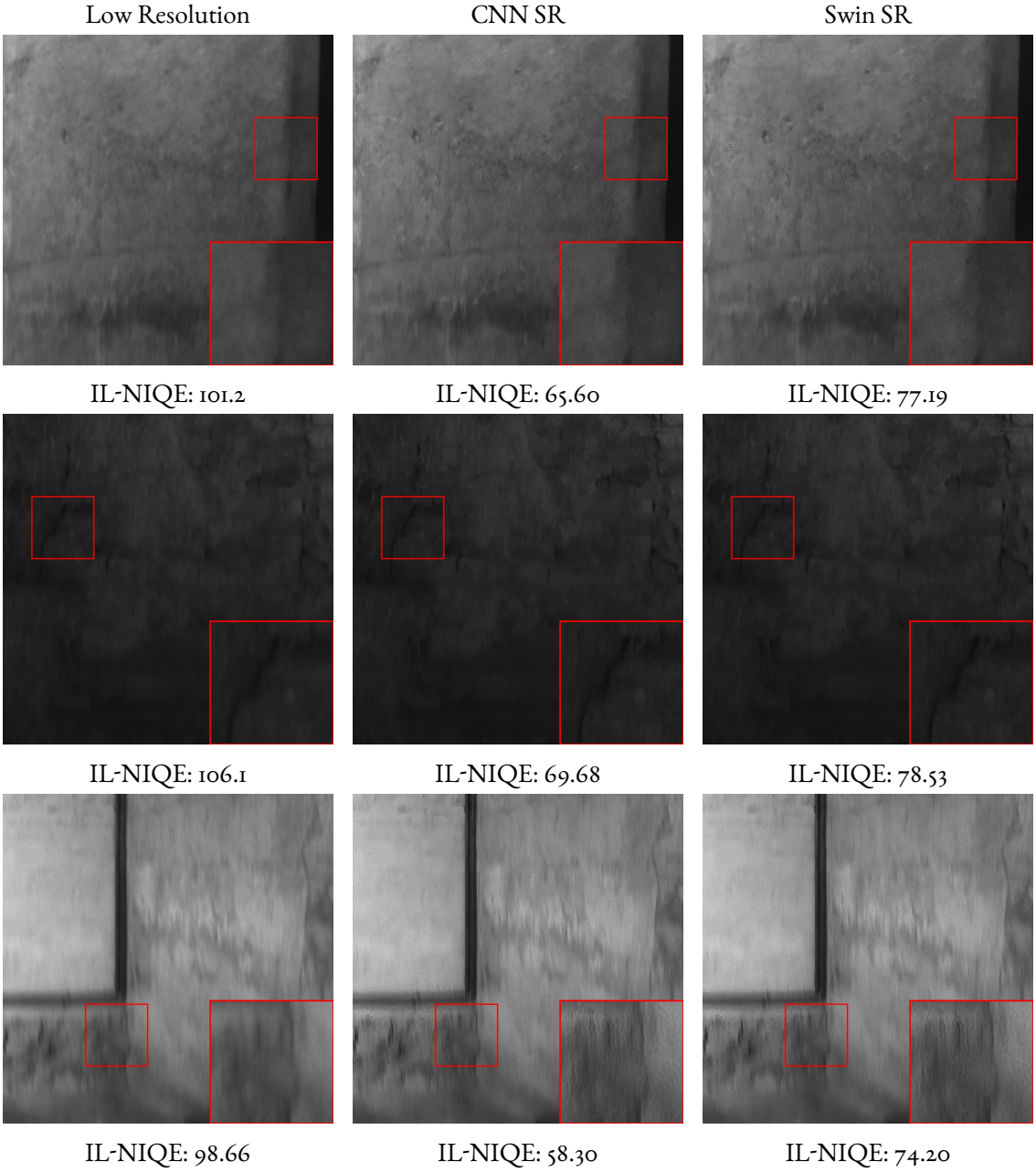


Figure 4.14: Example PANCAM image details and the SR images with our two architectures. At these angles our network struggle to super-resolve effectively.

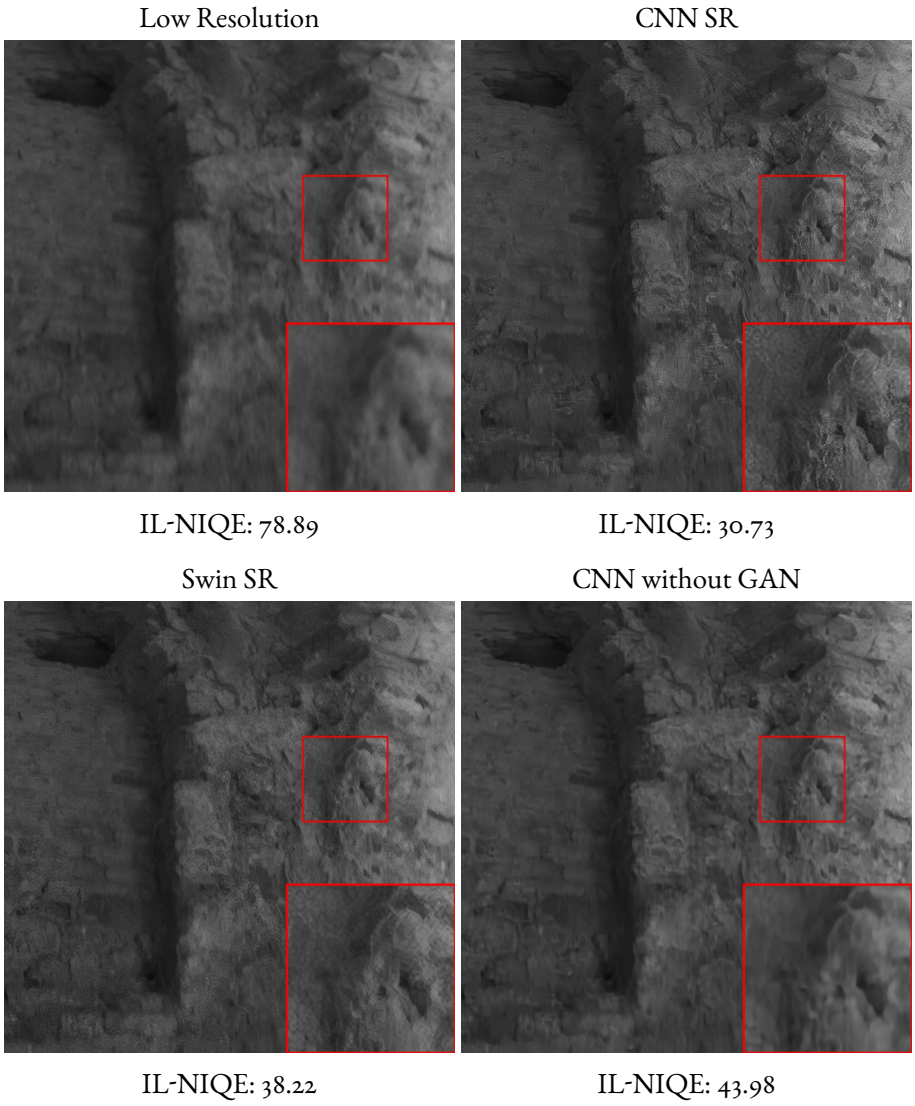
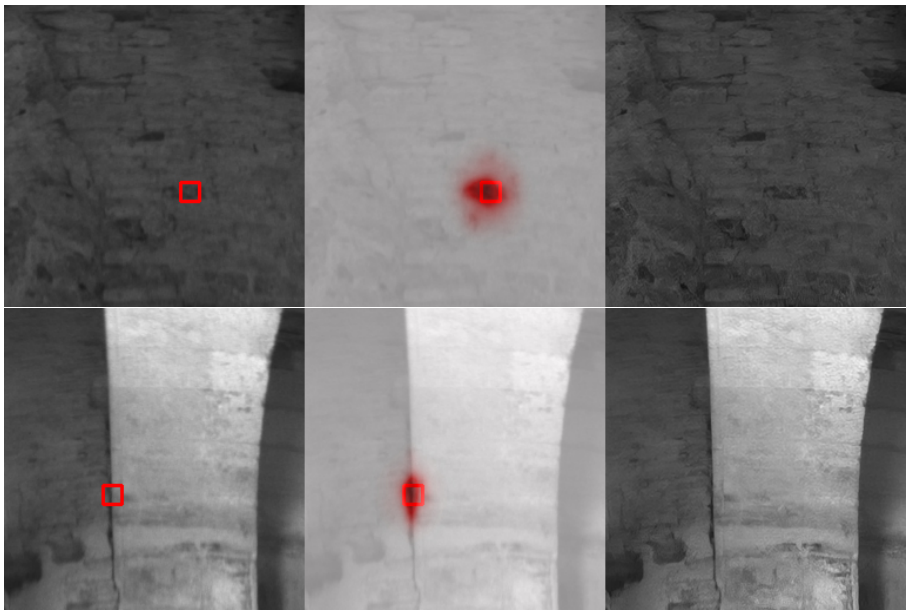
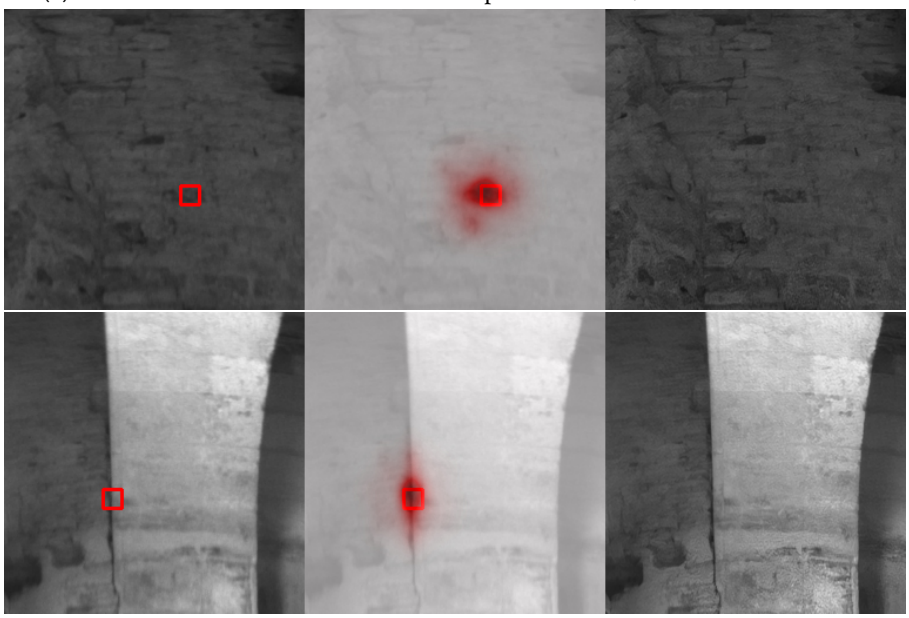


Figure 4.15: Comparison of different SR architectures on a LR image that tends to produce artifacts. Note that the CNN architecture trained without a discriminator, while achieving higher IL-NIQE scores and less sharpness, does not present the same unpleasant artifacts of the architecture trained with GAN.



(a) LAM for our CNN Architecture. Top DI= 2.076, Bottom DI= 1.405



(b) LAM for our Swin Architecture. Top DI= 4.019, Bottom DI= 3.152

Figure 4.16: LAM attribute for our two architectures, CNN (top) and Swin (bottom), in two example PANCAM images. Left is the LR input with the patch where gradients are computed highlighted, center is the KDE fit of LAM blended with the input, and right is the SR image. The DI scores confirm that the Swin architecture is able to take advantage of more information compared to the CNN backbone in the task of SR.

5 Conclusion

In this thesis we analyzed the problem of single image Super Resolution from an approach based on neural networks trained in a supervised manner, building tasks of increasing difficulty to attempt the SR of the hyperpanoramic images captured by PANCAM [3]. From the results showcased in the last chapter, one can see that in the simple task of classical SR our neural networks achieve very good results, but as more complicated degradations come into play they begin to struggle and are not always able to reconstruct the images properly. In particular with the spatially variant degradations of PANCAM images, our architectures performed well at zenith angles where the blur was not strong, but were not able to reconstruct the more complicated degradations. However, given their general performance and superiority compared to other methods, neural networks are a highly effective solution to the problem of SR.

We have also highlighted the advantages and disadvantages of training an adversarial network: a discriminator helps the generator to output more realistic and sharper images, but it is more susceptible to the creation of artifacts. Striking a balance between the two is not an easy task, especially as we have seen that the no-reference image quality metric we used was not able to take into account in its scores the presence of artifacts, and thus we still lacked a consistent way of evaluating the performance of our networks on the PANCAM images. However, if the specific use-case is of scientific nature, the presence of artifacts becomes unacceptable, and adversarial training should be avoided in favor of a more straightforward approach.

We also highlighted the differences between the two different architectures we employed: convolutional networks are generally more optimized and thus faster than Transformers (while also having lower memory requirements) but are in general outperformed by them. As explained by using Local Attention Maps [1], it seems that Transformers are in fact able to take advantage of more spatial information and have a wider receptive field, and this should be the reason behind their higher performance. However, in a situation where a large amount of images need to be processed in a relative short time or if the network needs to be deployed in device with low performant hardware, a convolutional architecture should be preferred.

As also said above, we have also highlighted some of the ineffectiveness of the ordinary validation metrics that we have used and the problems of using a no-reference approach such as IL-NIQE. Specifically for PANCAM images, it is also difficult to quantify how much information is really gained by SR or how effective the reconstruction of the HR image is.

While we have proposed some possible improvements to our approach in 4.7, we believe the focus should mainly be in taking advantage of the local information which encodes the spatially variant nature of the degradation (as we attempted to do with equation 4.12), so the next step would be to implement an architecture similar to [40] that takes advantage of deformable convolutions, and properly select the hyperparameters of the degradation pipeline in a more objective manner.

References

[1] J. Gu and C. Dong, "Interpreting super-resolution networks with local attribution maps," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 9199–9208.

[2] J. D. Simpkins and R. L. Stevenson, "An introduction to super-resolution imaging," in *Mathematical Optics*. CRC Press, 2018, pp. 555–580.

[3] C. Pernechele, "Hyper hemispheric lens," *Optics express*, vol. 24, no. 5, pp. 5014–5019, 2016.

[4] X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-esrgan: Training real-world blind super-resolution with pure synthetic data," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 1905–1914.

[5] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10 012–10 022.

[6] A. Dosovitskiy, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[7] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," *Communications of the ACM*, vol. 63, no. 11, pp. 139–144, 2020.

[8] D. P. Kingma, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, http://www.deeplearningbook.org.

[10] B. Hanin, "Universal function approximation by deep neural nets with bounded width and relu activations," *Mathematics*, vol. 7, no. 10, p. 992, 2019.

[11] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural networks*, vol. 4, no. 2, pp. 251–257, 1991.

[12] D. P. Bertsekas and J. N. Tsitsiklis, "Gradient convergence in gradient methods with errors," *SIAM Journal on Optimization*, vol. 10, no. 3, pp. 627–642, 2000.

[13] A. Griewank and A. Walther, *Evaluating Derivatives*, 2nd ed. Society for Industrial and Applied Mathematics, 2008. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9780898717761

[14] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," 2018. [Online]. Available: https://arxiv.org/abs/1502.05767

[15] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019. [Online]. Available: https://arxiv.org/abs/1912.01703

[16] T. Dozat, "Incorporating nesterov momentum into adam," in *Proceedings of the 4th International Conference on Learning Representations*, 2016, pp. 1–4.

[17] J. Qin, J. Fang, Q. Zhang, W. Liu, X. Wang, and X. Wang, "Resizemix: Mixing data with preserved object information and true labels," 2020. [Online]. Available: https://arxiv.org/abs/2012.11101

[18] C.-C. Hsu, C.-M. Lee, and Y.-S. Chou, "Drct: Saving image super-resolution away from information bottleneck," *arXiv preprint arXiv:2404.00722*, 2024.

[19] X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. Change Loy, "Esrgan: Enhanced super-resolution generative adversarial networks," in *Proceedings of the European conference on computer vision (ECCV) workshops*, 2018, pp. 0–0.

[20] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 1874–1883.

[21] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016. [Online]. Available: https://arxiv.org/abs/1607.06450

[22] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[23] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part II 14*. Springer, 2016, pp. 694–711.

[24] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: https://arxiv.org/abs/1409.1556

[25] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.

[26] L. Zhang, L. Zhang, and A. C. Bovik, "A feature-enriched completely blind image quality evaluator," *IEEE Transactions on Image Processing*, vol. 24, no. 8, pp. 2579–2591, 2015.

[27] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," *arXiv preprint arXiv:1802.05957*, 2018.

[28] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *International conference on machine learning*. PMLR, 2013, pp. 1139–1147.

[29] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[30] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1026–1034.

[31] C. E. Shannon, "Communication in the presence of noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949.

[32] C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE transactions on pattern analysis and machine intelligence*, vol. 38, no. 2, pp. 295–307, 2015.

[33] C. Ledig, L. Theis, F. Huszar, J. Caballero, A. Cunningham, A. Acosta *et al.*, "Photo-realistic single image super-resolution using a generative adversarial network [c]," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, vol. 2017, 2017, pp. 4681–4690.

[34] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte, "Swinir: Image restoration using swin transformer," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 1833–1844.

[35] E. Simioni, C. Pernechele, W. Erb, A. Marchini, P. Martini, L. Lessio, L. Penasa, and M. Beghini, "A-central model for the geometric calibration of hyper-hemispherical lenses," *Opt. Express*, vol. 32, no. 20, pp. 34 777–34 795, Sep 2024. [Online]. Available: https://opg.optica.org/oe/abstract.cfm?URI=oe-32-20-34777

[36] D. Borrmann, A. Nüchter, A. Bredenbeck, J. Zevering, F. Arzberger, C. Reyes Mantilla, A. P. Rossi, F. Maurelli, V. Unnithan, H. Dreger *et al.*, "Lunar caves exploration with the daedalus spherical robot," in *52nd Lunar and Planetary Science Conference*, no. 2548, 2021, p. 2073.

[37] NeoSR, "NeoSR: Open-source framework for training super-resolution models," https://github.com/neosr-project/neosr, 2023.

[38] M. Bevilacqua, A. Roumy, C. Guillemot, and M. Alberi-Morel, "Low-complexity single-image super-resolution based on nonnegative neighbor embedding," in *British Machine Vision Conference, BMVC 2012, Surrey, UK, September 3-7, 2012*, R. Bowden, J. P. Collomosse, and K. Mikolajczyk, Eds. BMVA Press, 2012, pp. 1–10. [Online]. Available: https://doi.org/10.5244/C.26.135

[39] E. Agustsson and R. Timofte, "Ntire 2017 challenge on single image super-resolution: Dataset and study," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, July 2017.

[40] F. Yu, X. Wang, M. Cao, G. Li, Y. Shan, and C. Dong, "Osrt: Omnidirectional image super-resolution with distortion-aware transformer," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 13 283–13 292.

[41] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," 2018. [Online]. Available: https://arxiv.org/abs/1710.09412

[42] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "Cutmix: Regularization strategy to train strong classifiers with localizable features," 2019. [Online]. Available: https://arxiv.org/abs/1905.04899

[43] J. Yoo, N. Ahn, and K.-A. Sohn, "Rethinking data augmentation for image super-resolution: A comprehensive analysis and a new strategy," 2020. [Online]. Available: https://arxiv.org/abs/2004.00448

[44] C. Wan, H. Yu, Z. Li, Y. Chen, Y. Zou, Y. Liu, X. Yin, and K. Zuo, "Swift parameter-free attention network for efficient super-resolution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 6246–6256.

[45] L. Yang, R.-Y. Zhang, L. Li, and X. Xie, "Simam: A simple, parameter-free attention module for convolutional neural networks," in *International conference on machine learning*. PMLR, 2021, pp. 11 863–11 874.

[46] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*. Springer, 2015, pp. 234–241.

[47] R. Zeyde, M. Elad, and M. Protter, "On single image scale-up using sparse-representations," in *Curves and Surfaces: 7th International Conference, Avignon, France, June 24-30, 2010, Revised Selected Papers 7*. Springer, 2012, pp. 711–730.

[48] X. Wang, L. Xie, K. Yu, K. C. Chan, C. C. Loy, and C. Dong, "BasicSR: Open source image and video restoration toolbox," https://github.com/XPixelGroup/BasicSR, 2022.

[49] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *International conference on machine learning*. PMLR, 2017, pp. 3319–3328.

[50] A. Azzalini and A. D. Valle, "The multivariate skew-normal distribution," *Biometrika*, vol. 83, no. 4, pp. 715–726, 1996.

[51] L. Jiang, B. Dai, W. Wu, and C. C. Loy, "Focal frequency loss for image reconstruction and synthesis," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 13 919–13 929.

[52] J. Dai, H. Qi, Y. Xiong, Y. Li, G. Zhang, H. Hu, and Y. Wei, "Deformable convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 764–773.

Acknowledgments

I would like to thank Emanuele Simioni and Claudio Pernechele from INAF for welcoming me to their team and introducing me to the PANCAM and Daedalus Projects, and for the many helpful discussions during my internship there.

I thank my advisor Professor Erb for his continued support throughout the last months of my studies. I also thank the Dept. of Mathematics for providing the computing facilities that were used for this work.

A special thanks goes to my mother Elena, who has given me the strength, and the freedom, to figure myself out.